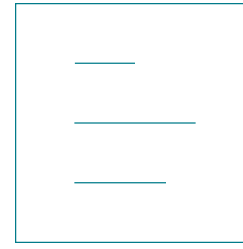


Support Vector Machines

by Daniel Meister, CTO at Datahouse



Support Vector Machines

Agenda

Motivation

A constructed example to "compare" with linear regression

Introduction

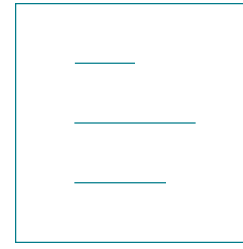
Why is it called «Support Vector Machine» incl. short theory introduction

Classification Problems

Some examples for classification problems; parameter tuning

Support Vector Regression

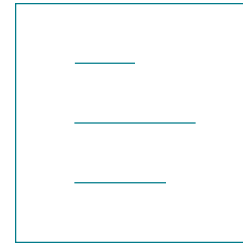
A short support vector regression lab



Support Vector Machines

Material and Sources

- obviously this presentation takes inspiration and material from various sources
- many of them are itself interesting articles and papers to read
- at the end of the slide set there will be a collection of links for further reading



Support Vector Machines

Organizational Points

- the slides and the accompanying lecture contain all the main ideas
- you should try to follow the more practical examples by executing the R code yourself
- there are also a few ideas for small projects/tasks to do more detailed work
 - if you complete one of those and would like some feedback feel free to send it to me
 - preferably as an RMarkdown file in a Git repository (but I can also handle Word files via e-mail)
- this is a fairly new lecture so there is (not yet) much additional material
 - also if you have feedback to share please let me know!

Technical Notes

Technical Notes

Base R and the Tidyverse

- We use Tidyverse functionality in many projects
- So you will also find some in the examples
- I'll try not to use the most complex functions so the code remains readable

```
mtcars %>%  
  names() %>%  
  purrr::map(~ count(mtcars, .data[[.x]]))
```



Motivation

Motivation

"Compare" to linear regression (1)

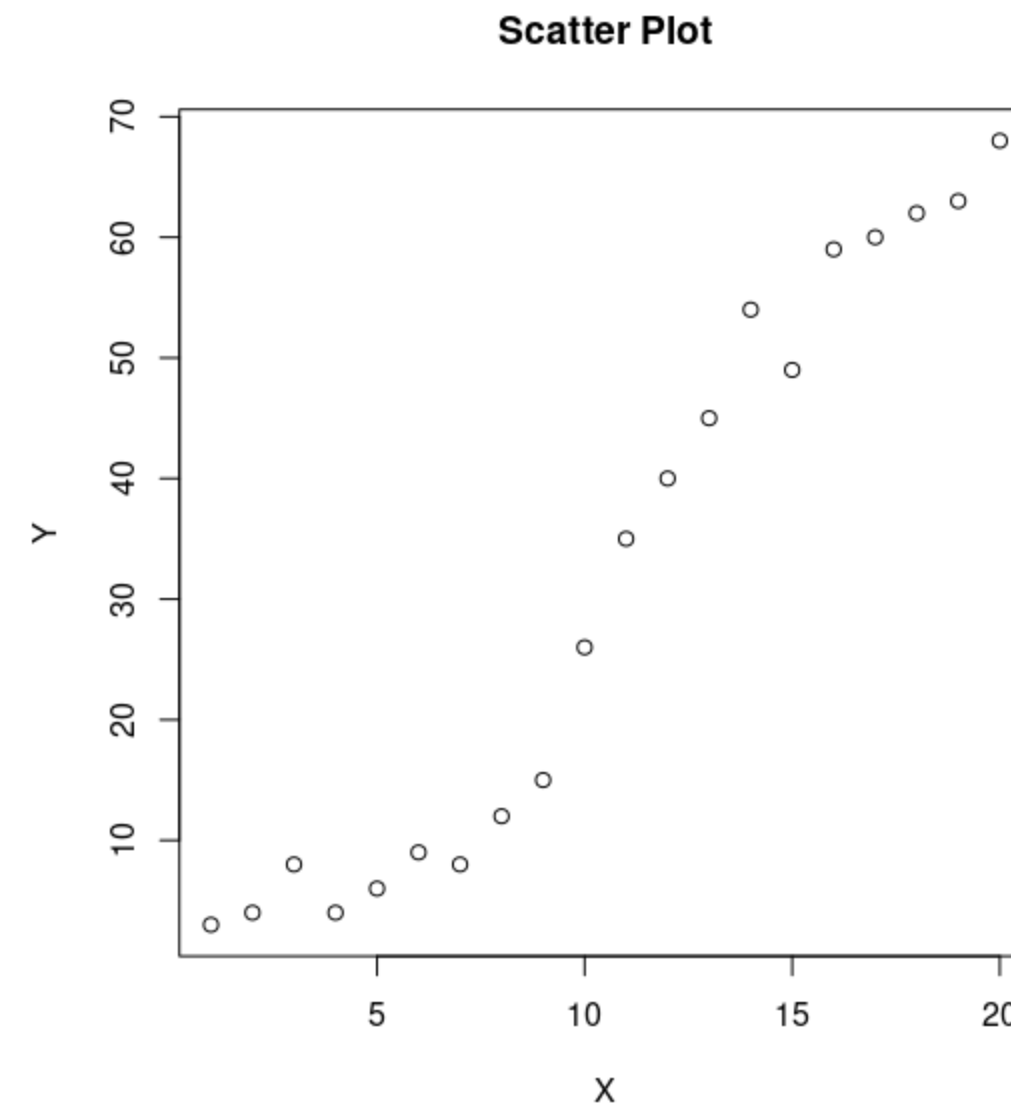
Source: [adapted from Alexandre Kowalczyk](#)

```
library(tidyverse)

# Load a list of a few 2D points
data <- read_csv("regression.csv")
data %>% head()

# A tibble: 6 x 2
      X     Y
  <dbl> <dbl>
1     1     3
2     2     4
3     3     8
4     4     4
5     5     6
6     6     9

# Scatter Plot
plot(data, main = "Scatter Plot")
```



Motivation

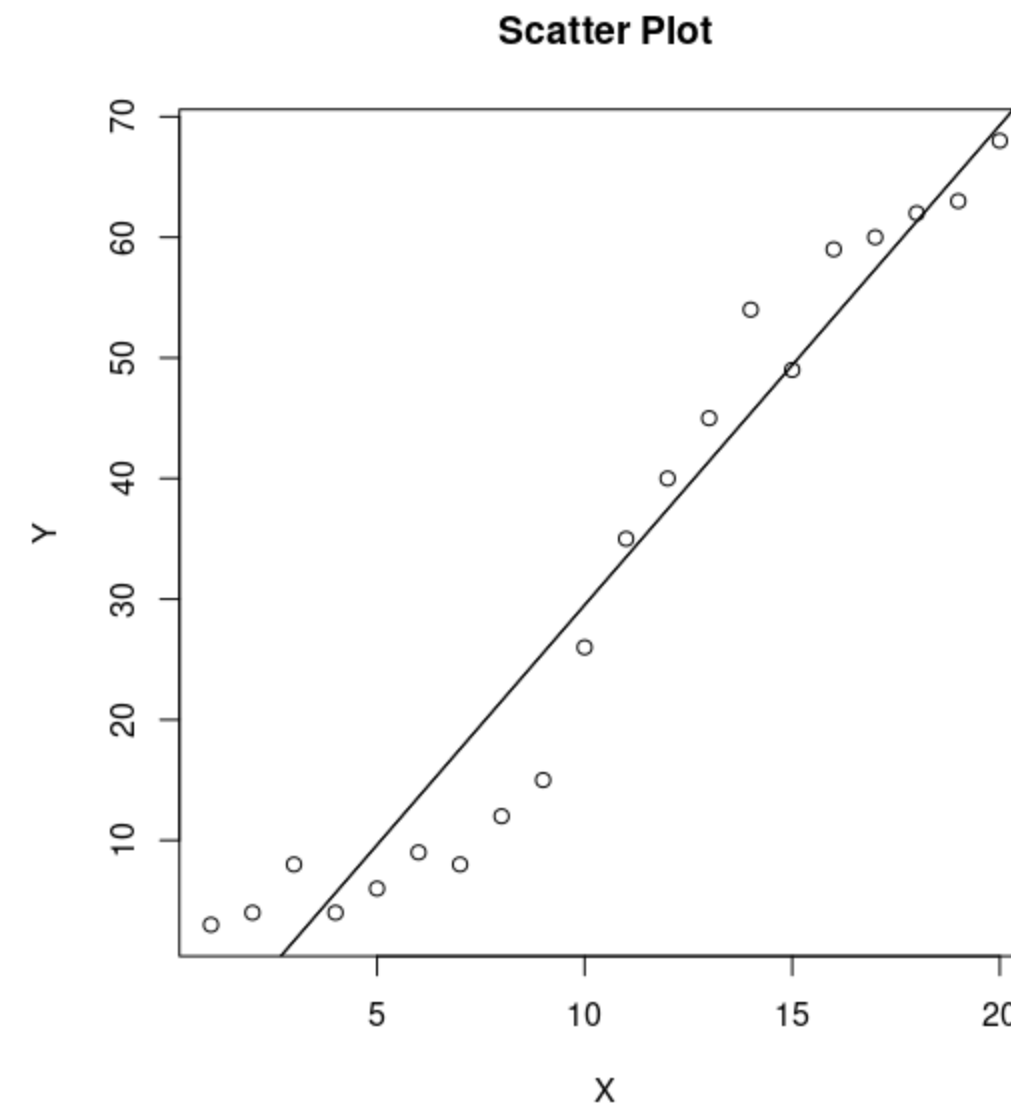
"Compare" to linear regression (2)

```
## Add best-fit line to the scatter plot
```

```
# Fit linear model using OLS  
model <- lm(Y~X, data)  
abline(model)
```

```
rmse <- data %>%  
  mutate(predictions = predict(model, .)) %>%  
  summarise(sqrt(sum((predictions - Y)**2)/n()))  
rmse
```

- results in an RMSE of 5.7



Motivation

"Compare" to linear regression (3)

```
## Fit SVR model and visualize using scatter plot
library(e1071)

plot(data)

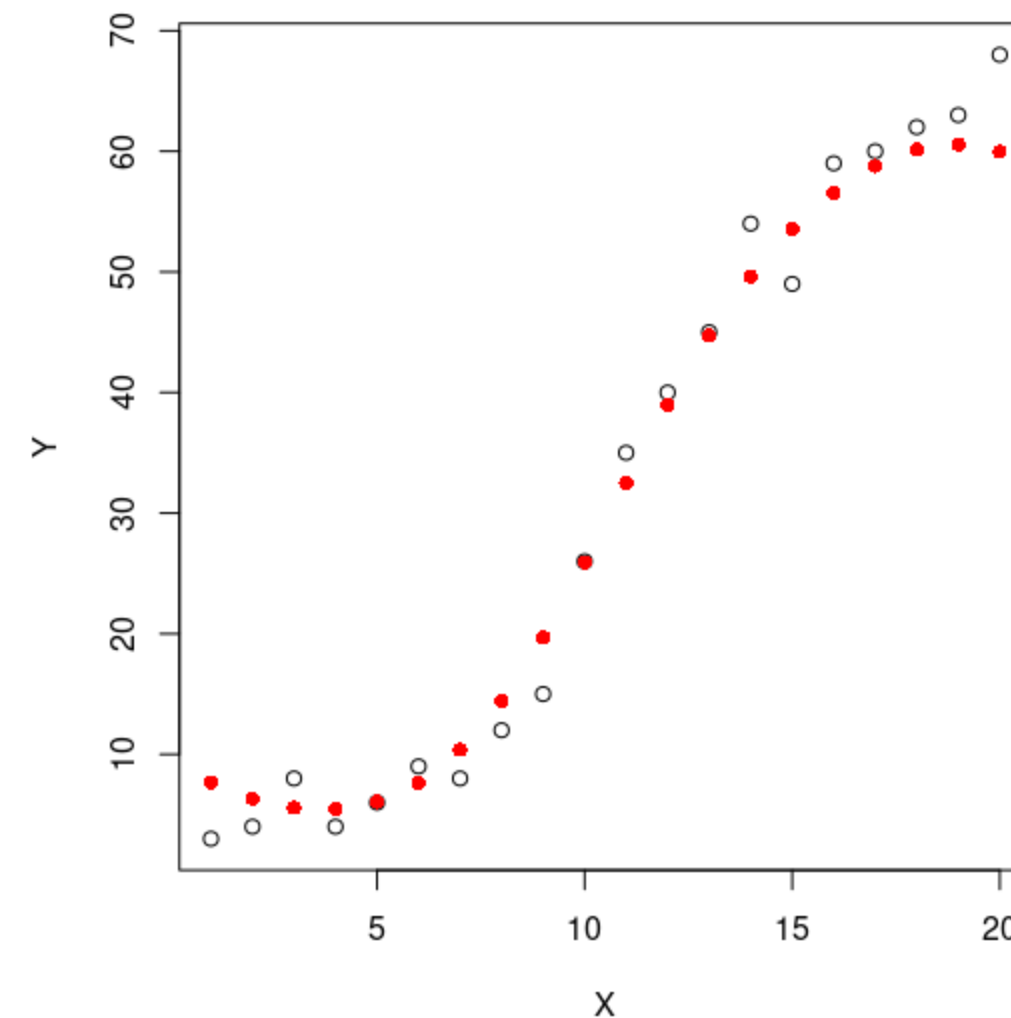
# Regression with SVM
modelsvm <- svm(Y ~ X, data)

# Predict using SVM regression
predYsvm <- predict(modelsvm, data)

# Overlay SVM Predictions on Scatter Plot
points(data$X, predYsvm, col="red", pch=16)

sqrt(sum(predYsvm - data$Y)**2/nrow(data))
```

- results in an RMSE of 1.2



Motivation

First Observations

- SVM models seems to be more "flexible"
 - think about whether this is only an advantage or what additional problems it could cause
- obviously there is some advanced logic going on in the background
- we could maybe also do a transform to get to a linear model

Introduction

Introduction

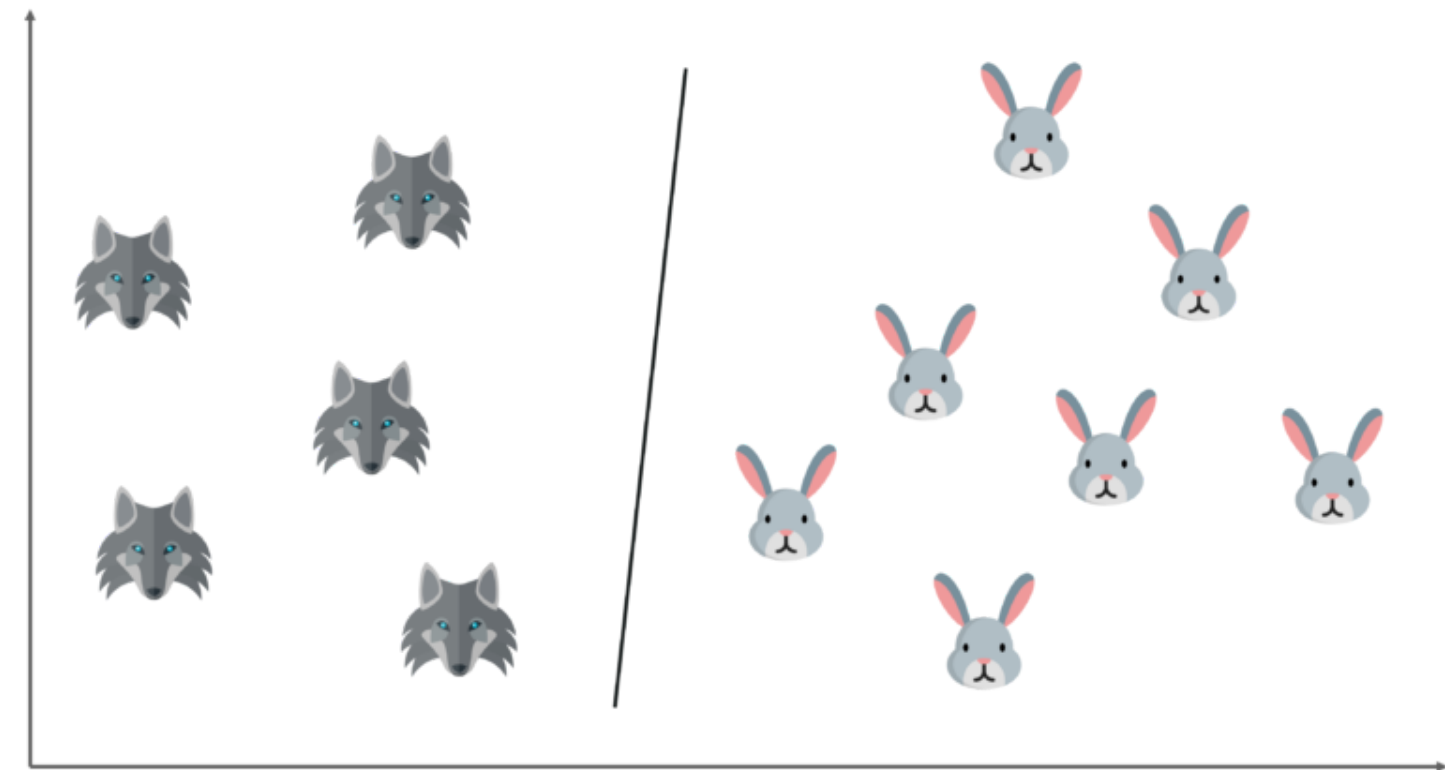
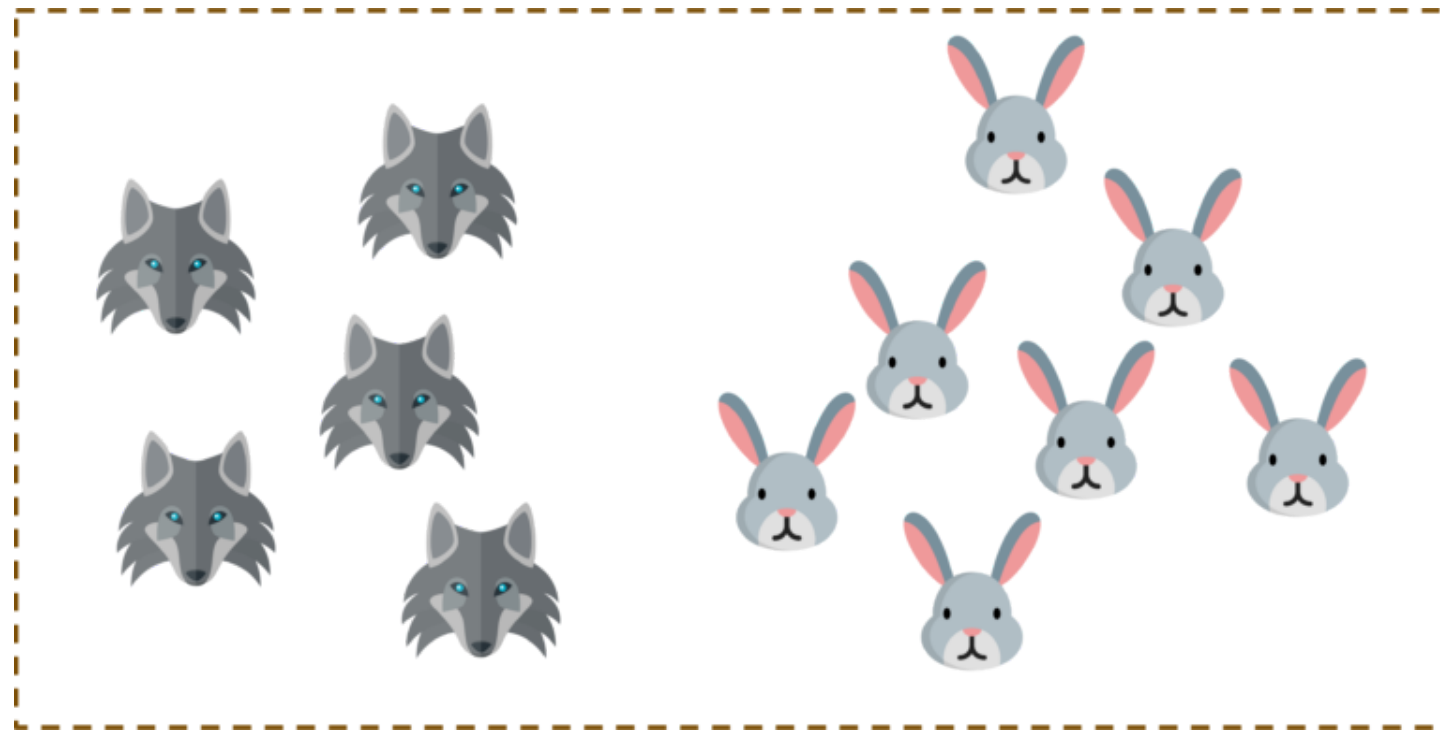
What is an SVM?

Definition on Wikipedia (slightly shortened)

In machine learning, support-vector machines (SVMs, also support-vector networks) are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis. Developed at AT&T Bell Laboratories by Vapnik with colleagues ..., it presents one of the most robust prediction methods, based on the statistical learning framework or VC theory Given a set of training examples, each marked as belonging to one or the other of two categories, an SVM training algorithm builds a model that assigns new examples to one category or the other, making it a non-probabilistic binary linear classifier (...). An SVM model is a representation of the examples as points in space, mapped so that the examples of the separate categories are divided by a clear gap that is as wide as possible. New examples are then mapped into that same space and predicted to belong to a category based on the side of the gap on which they fall.

Introduction

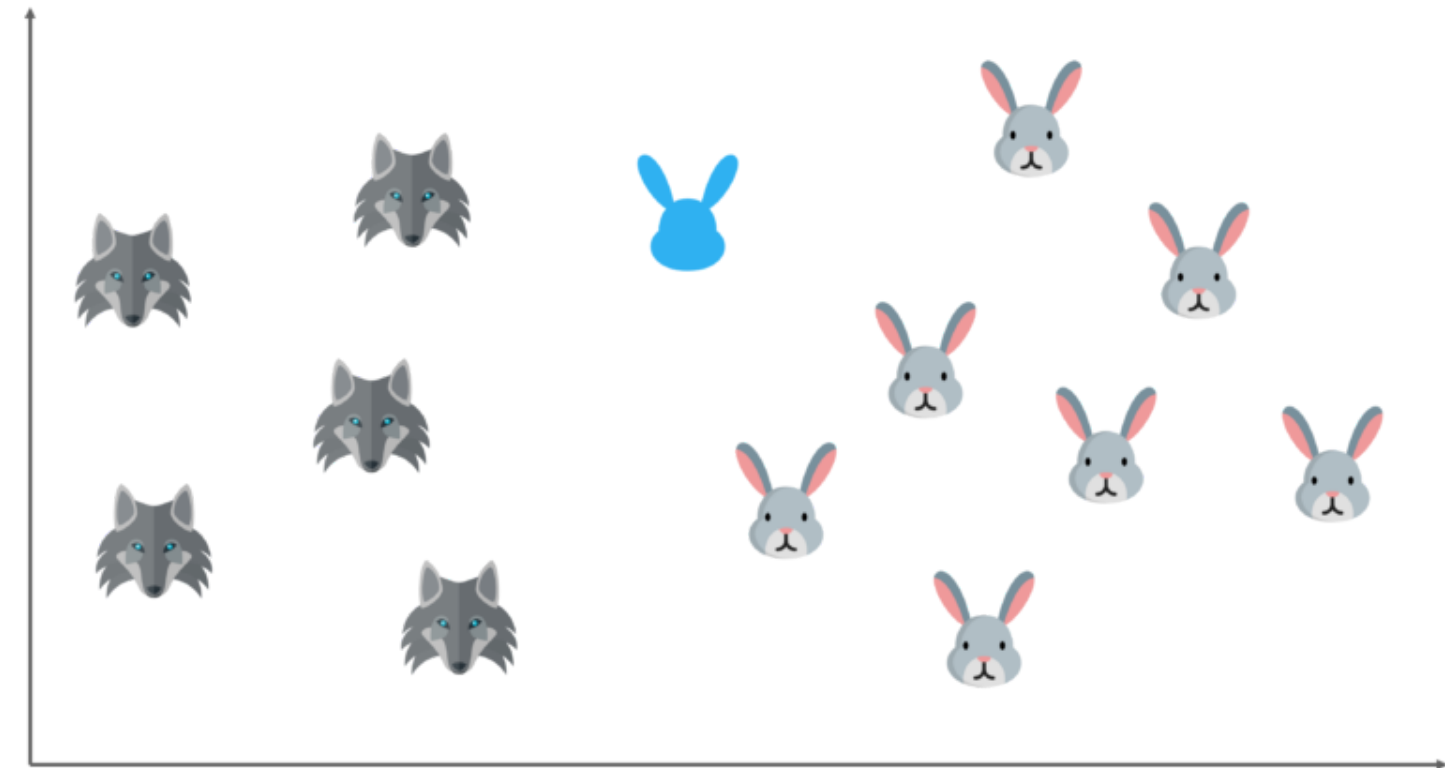
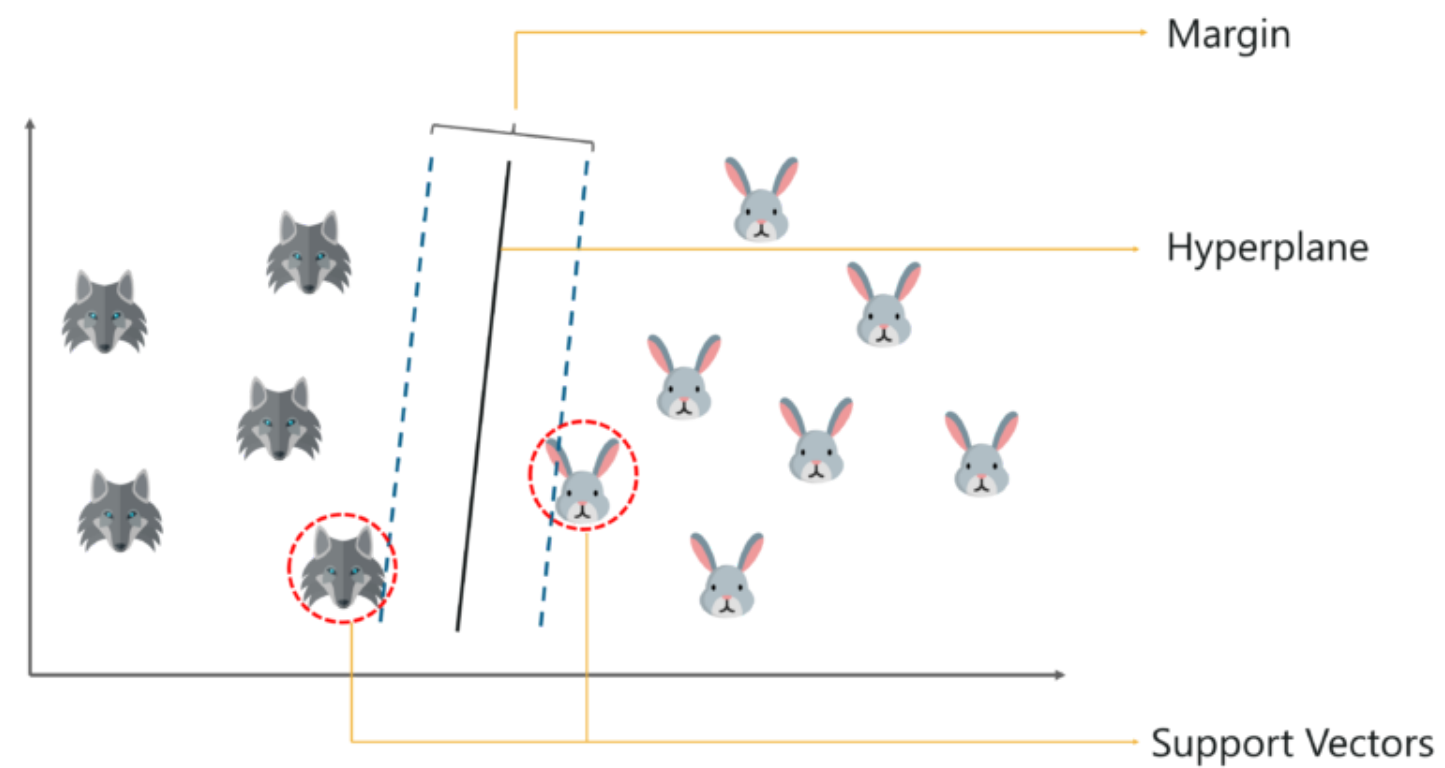
How does an SVM work? (1)



Source: [edureka.co](https://www.edureka.co)

Introduction

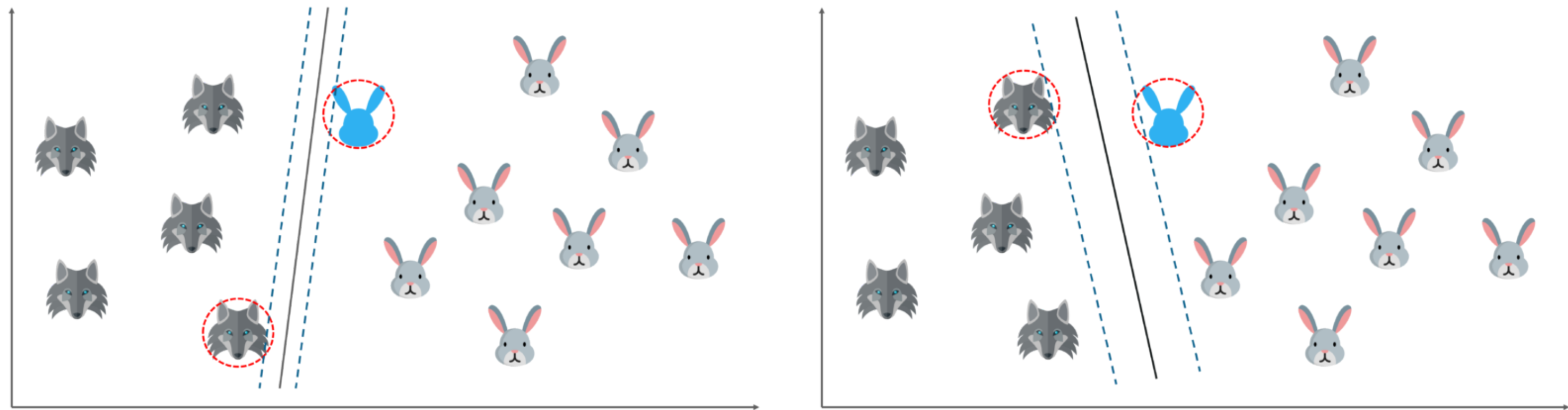
How does an SVM work? (2)



Source: [edureka.co](https://www.edureka.co)

Introduction

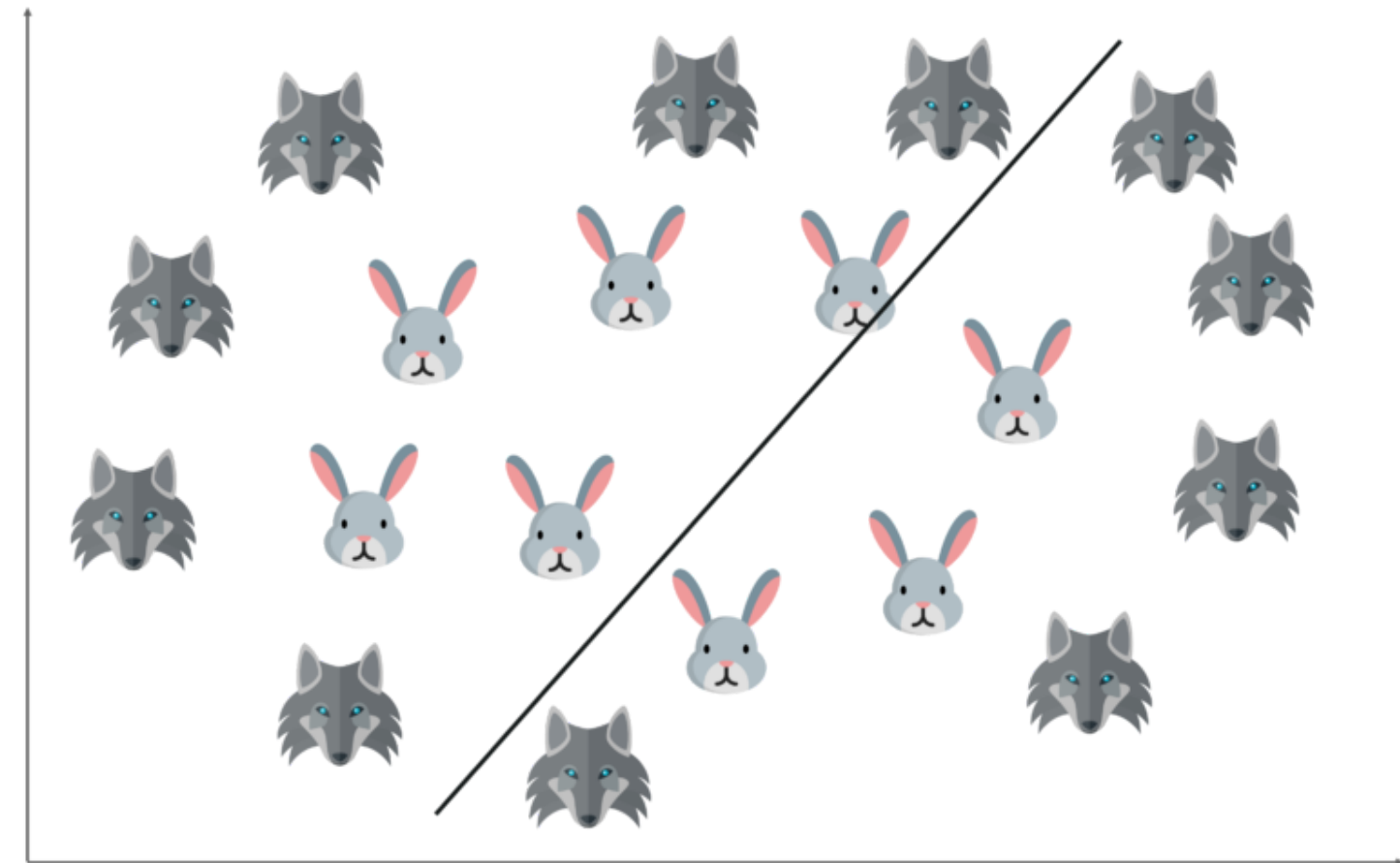
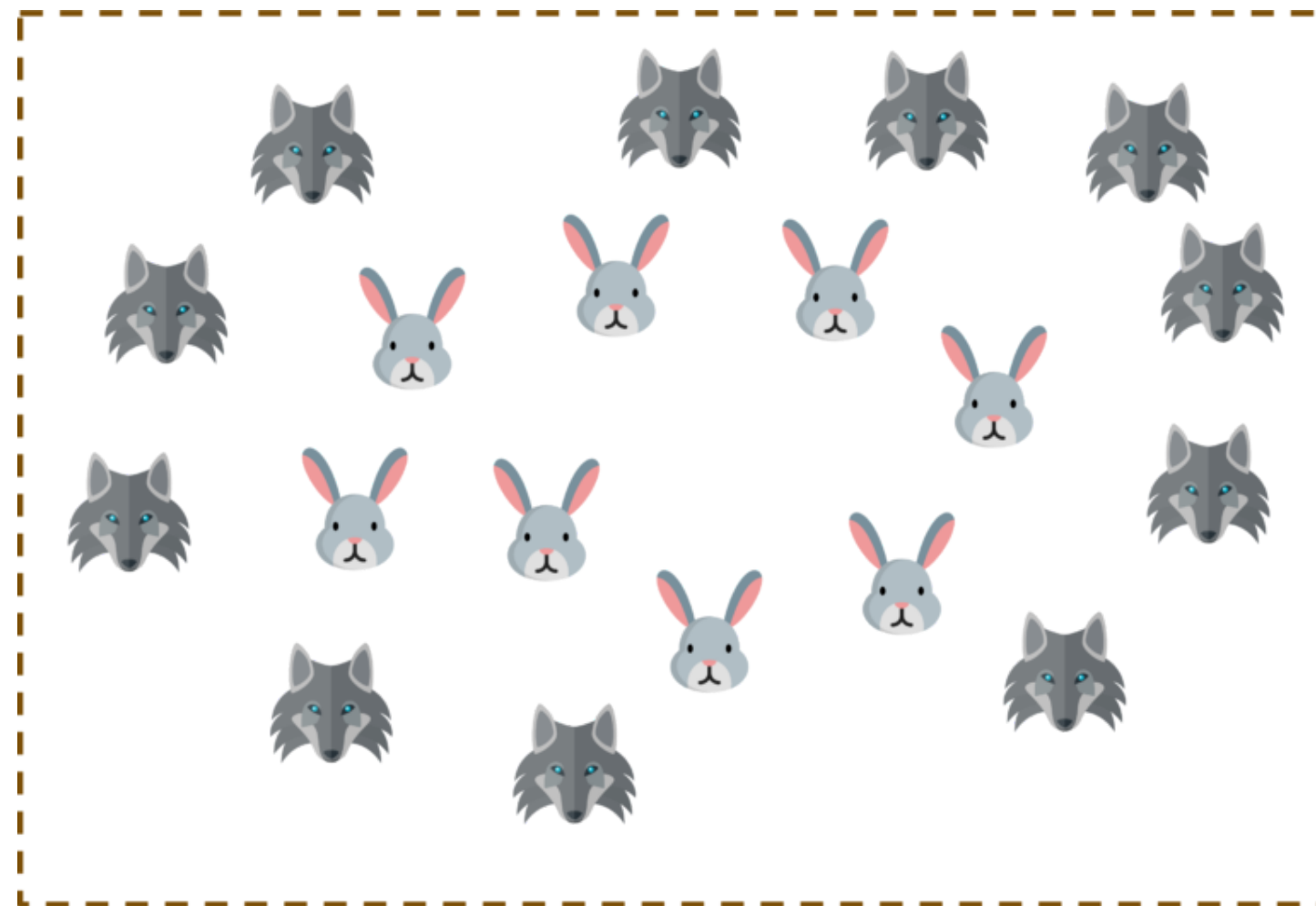
How does an SVM work? (3)



Source: [edureka.co](https://www.edureka.co)

Introduction

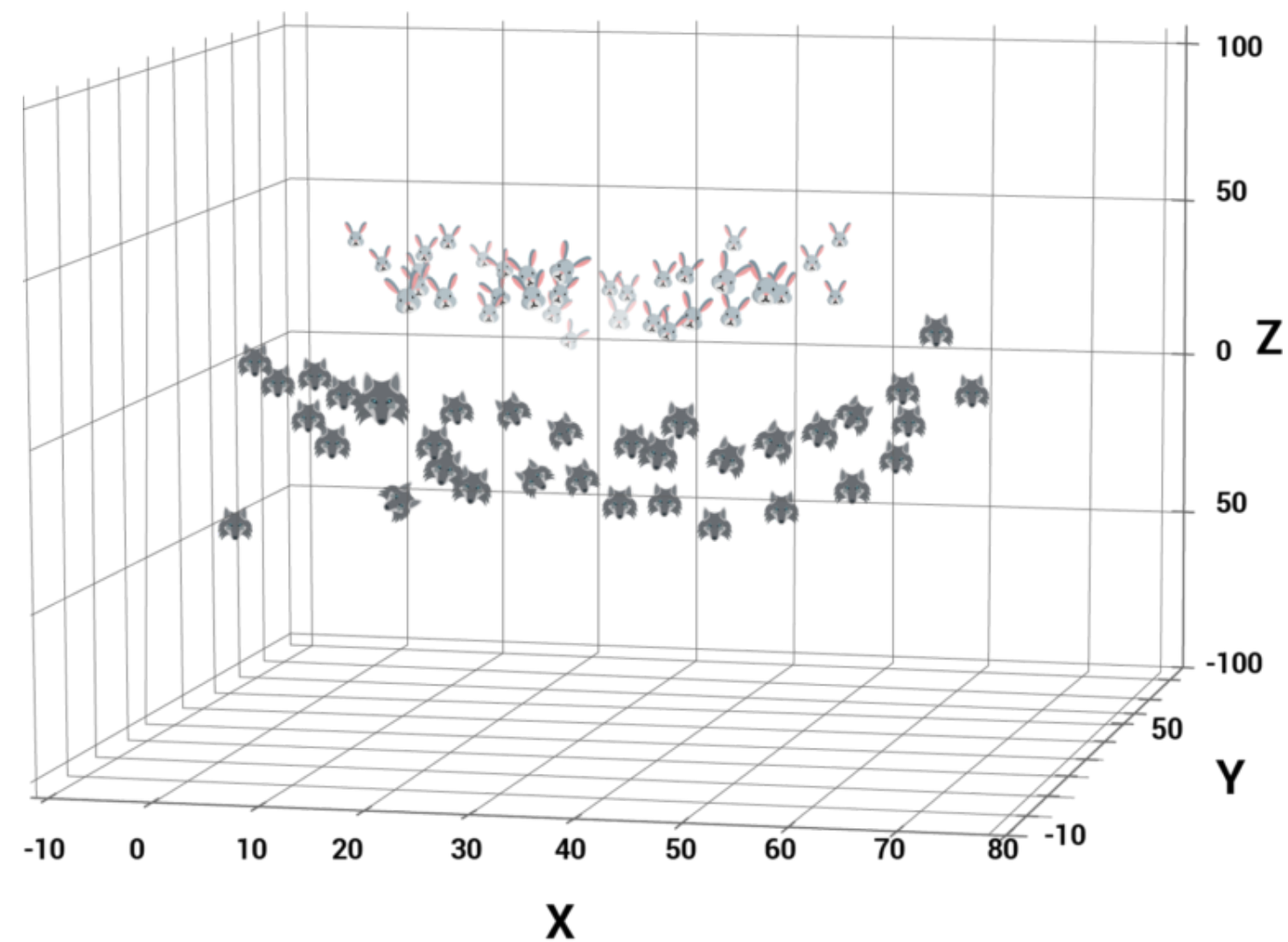
How does an SVM work? (4)



Source: [edureka.co](https://www.edureka.co)

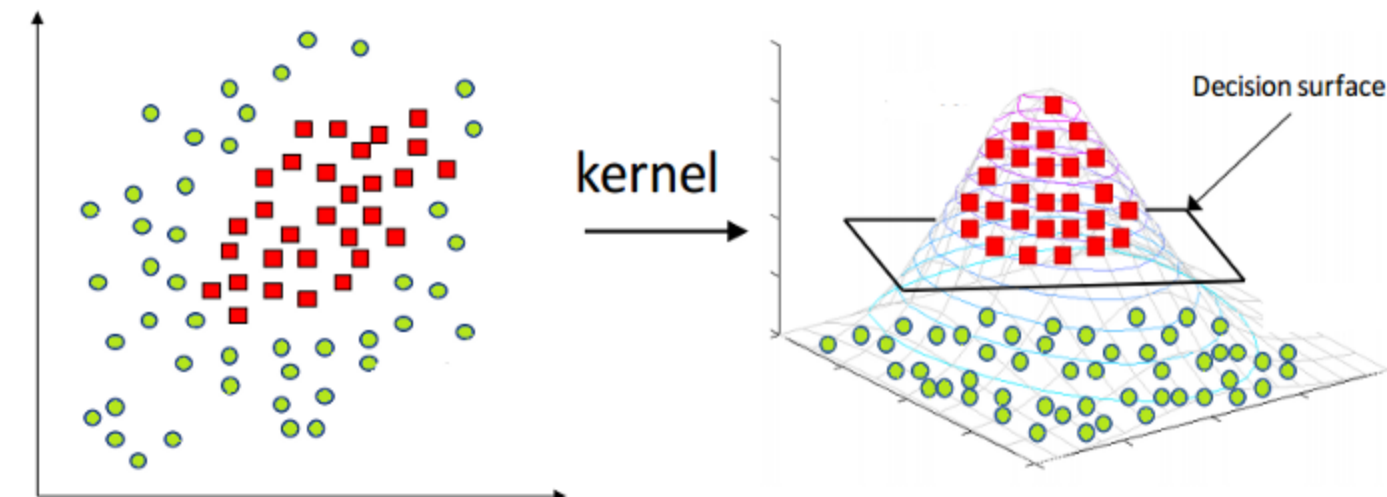
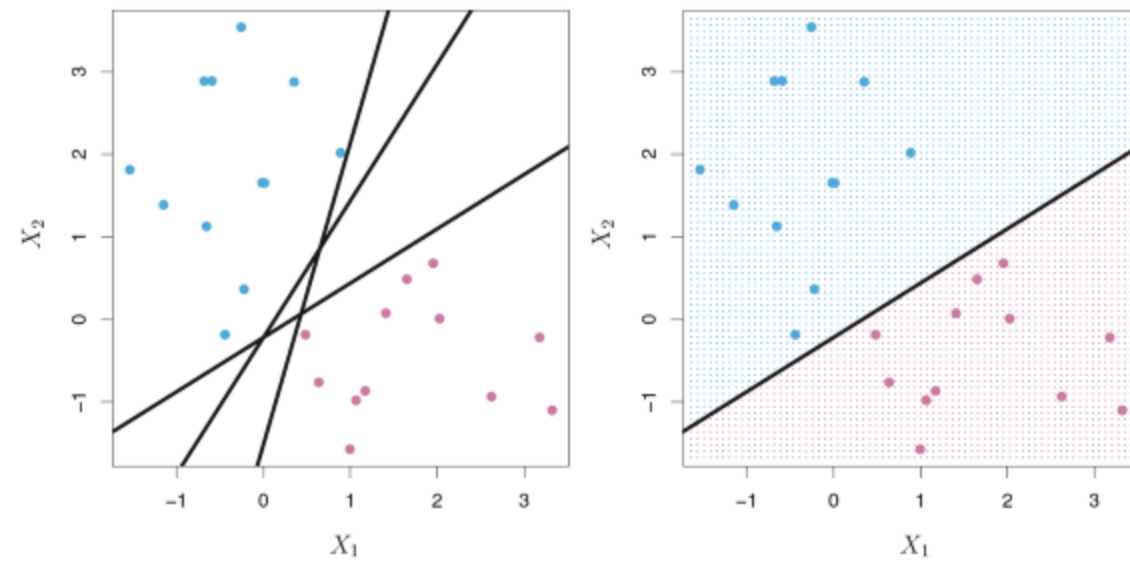
Introduction

How does an SVM work? (5)



Introduction

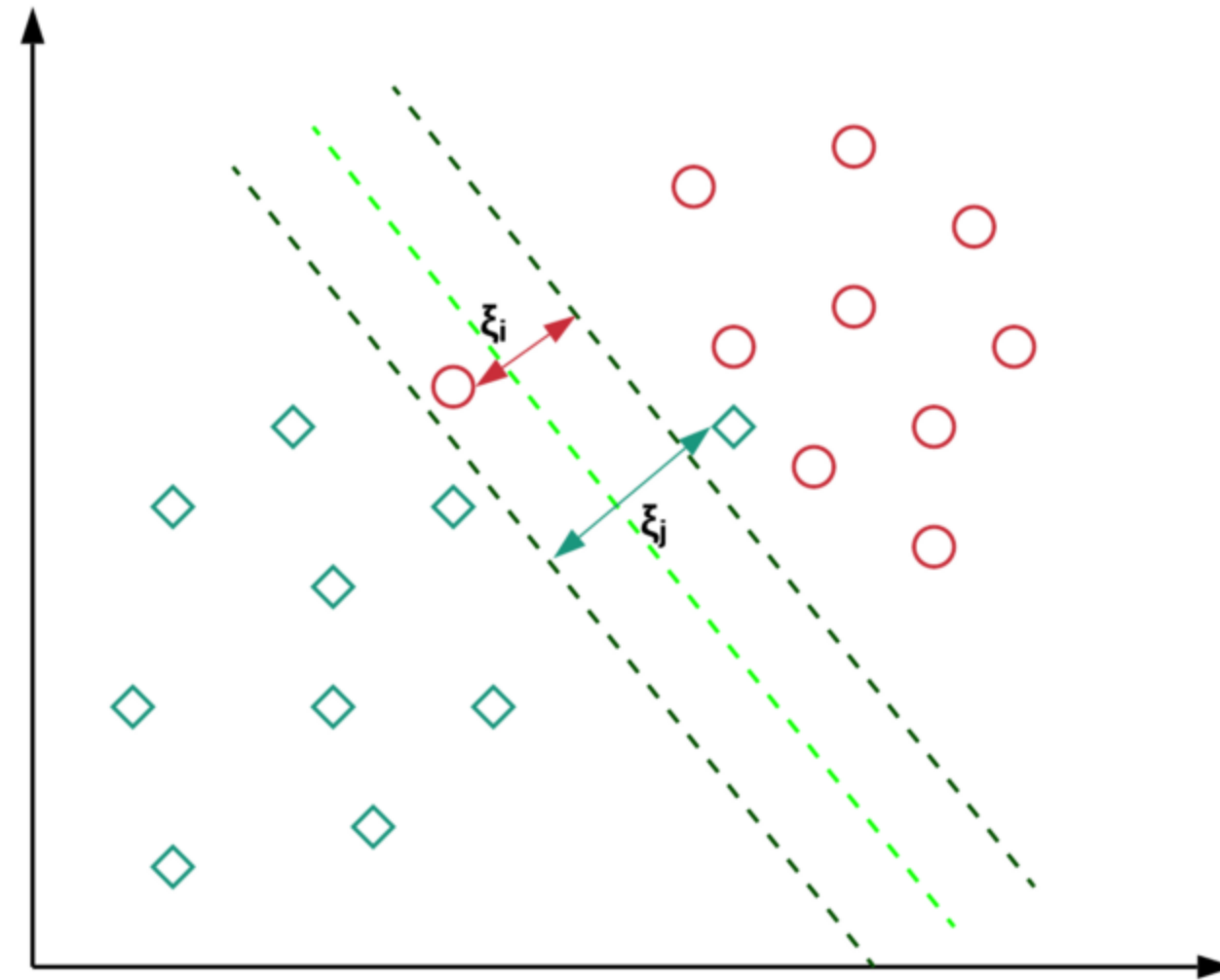
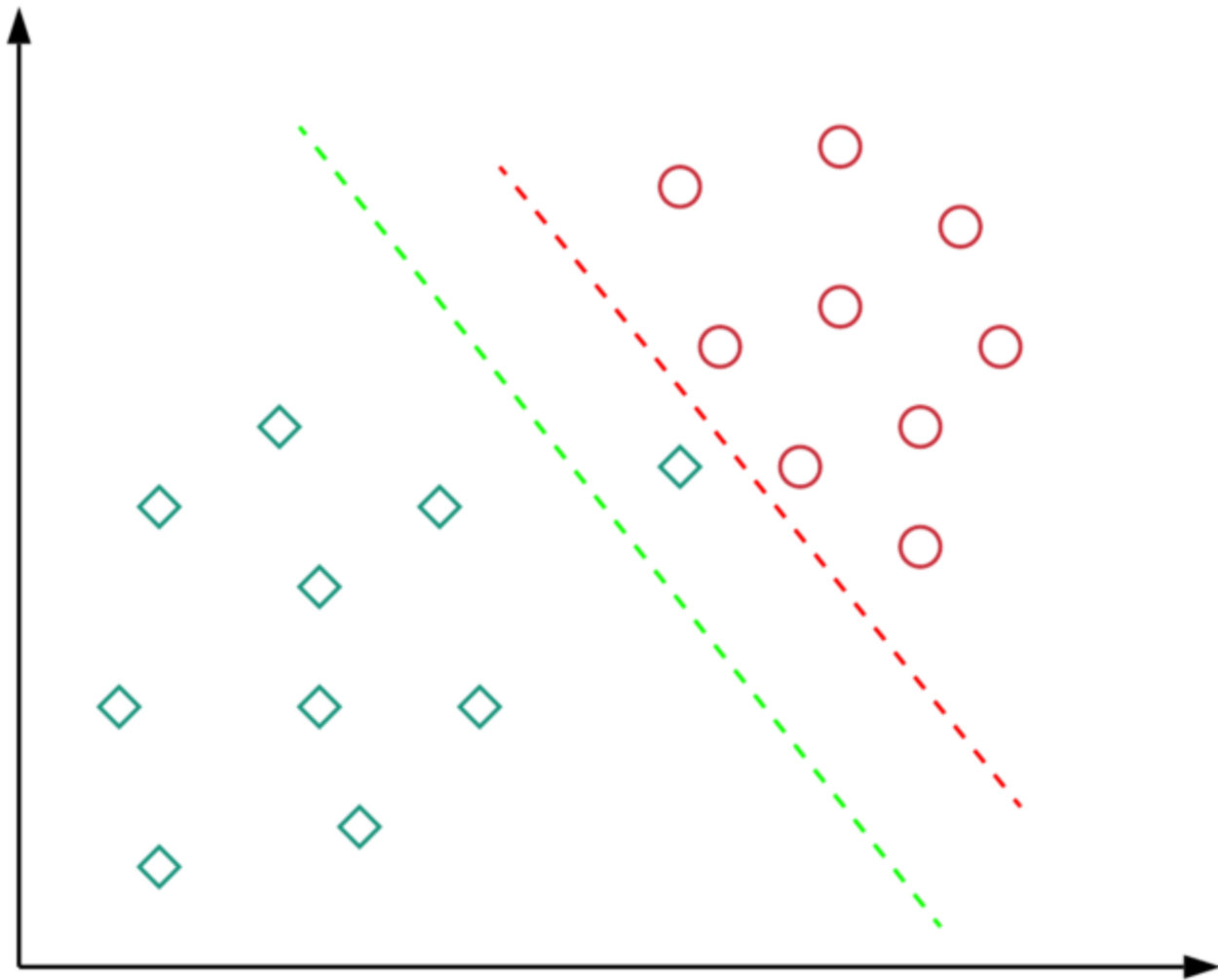
SVM Kernel Functions



Source: [medium.com](https://medium.com/@GrazeZhang), Graze Zhang

Introduction

Soft Margin Formulation

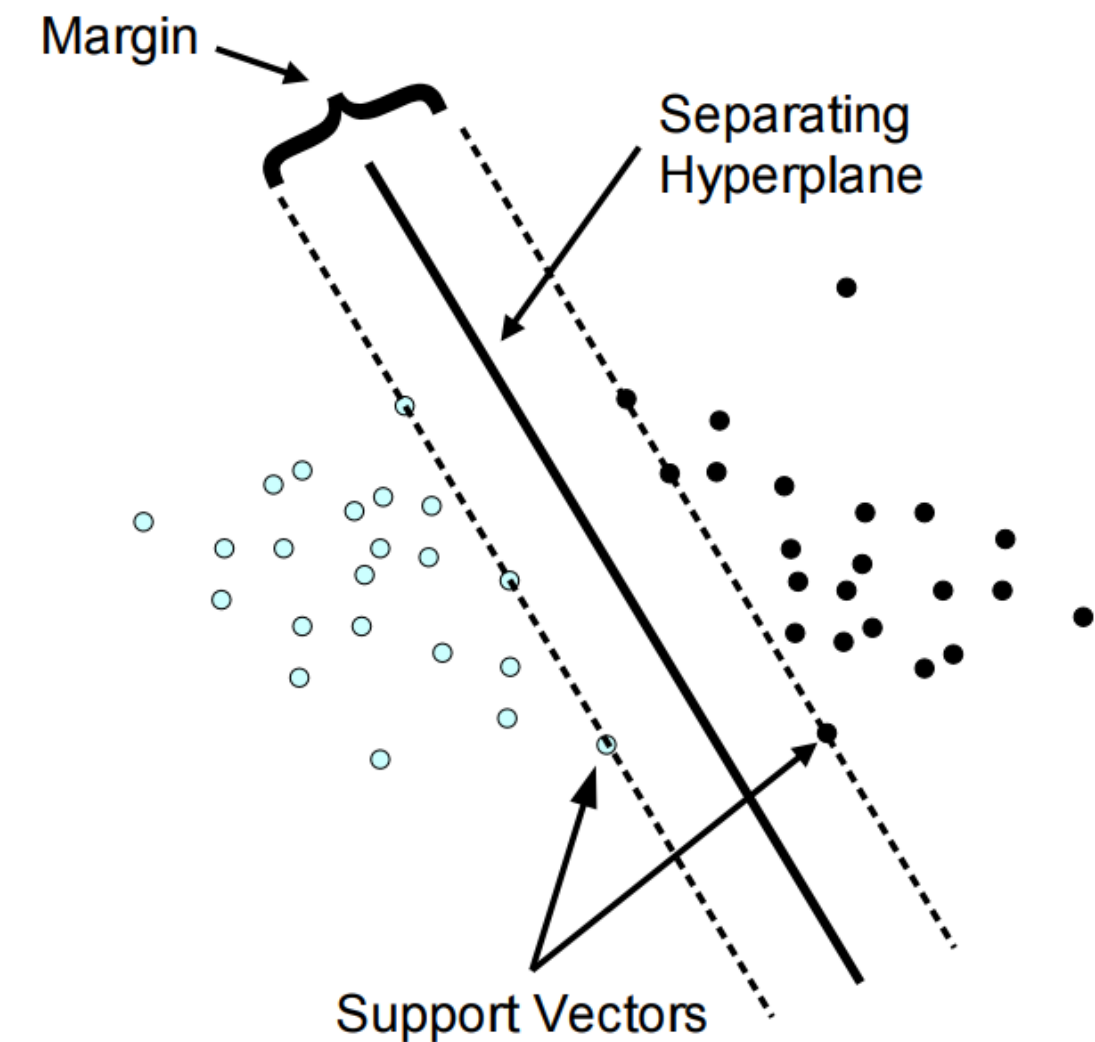


Introduction

SVM in R with e1071

Package Functionalities

- Class separation
- Overlapping classes
- Nonlinearity
- One-class-classification
- Multi-class classification
- epsilon-regression
- nu-classification/-regression



Introduction

Real World Applications

Face Detection

Using image training data, SVM classifies pixels in images like a face or non-face

Text Classification

Training data is used to categorize different types of documents and texts

Classifying Images

By classifying images with improved techniques, SVM increases search accuracy.

Bioinformatics

SVM algorithms have increased the effectiveness in protein homology detection, cancer classification, gene classification, and more.

Classification Problems

Classification Problems

Setup

The following examples are based on an adapted version of the [UC Business Analytics R Programming Guide, SVM](#)

Packages and seed

```
# set pseudorandom number generator
set.seed(10)

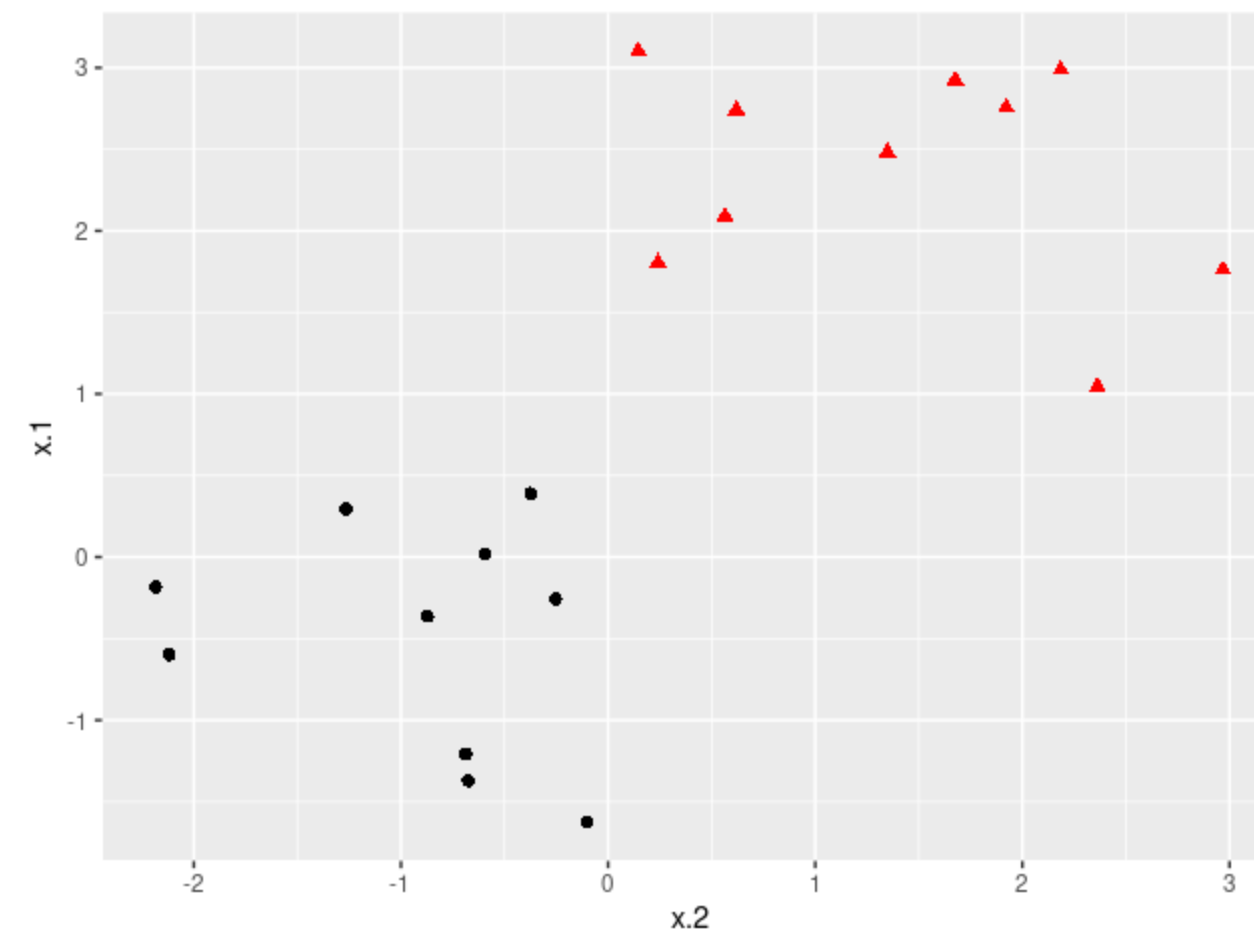
# Attach Packages
library(tidyverse) # data manipulation and visualization
library(kernlab)   # SVM methodology
library(e1071)     # SVM methodology
library(ISLR)      # contains example data set "Khan"
library(RColorBrewer) # customized coloring of plots
```


Classification Problems

Random Test Data (separated)

```
# Construct sample data set - completely separated
x <- matrix(rnorm(20*2), ncol = 2)
y <- c(rep(-1,10), rep(1,10))
x[y==1,] <- x[y==1,] + 2
dat <- data.frame(x=x, y=as.factor(y))

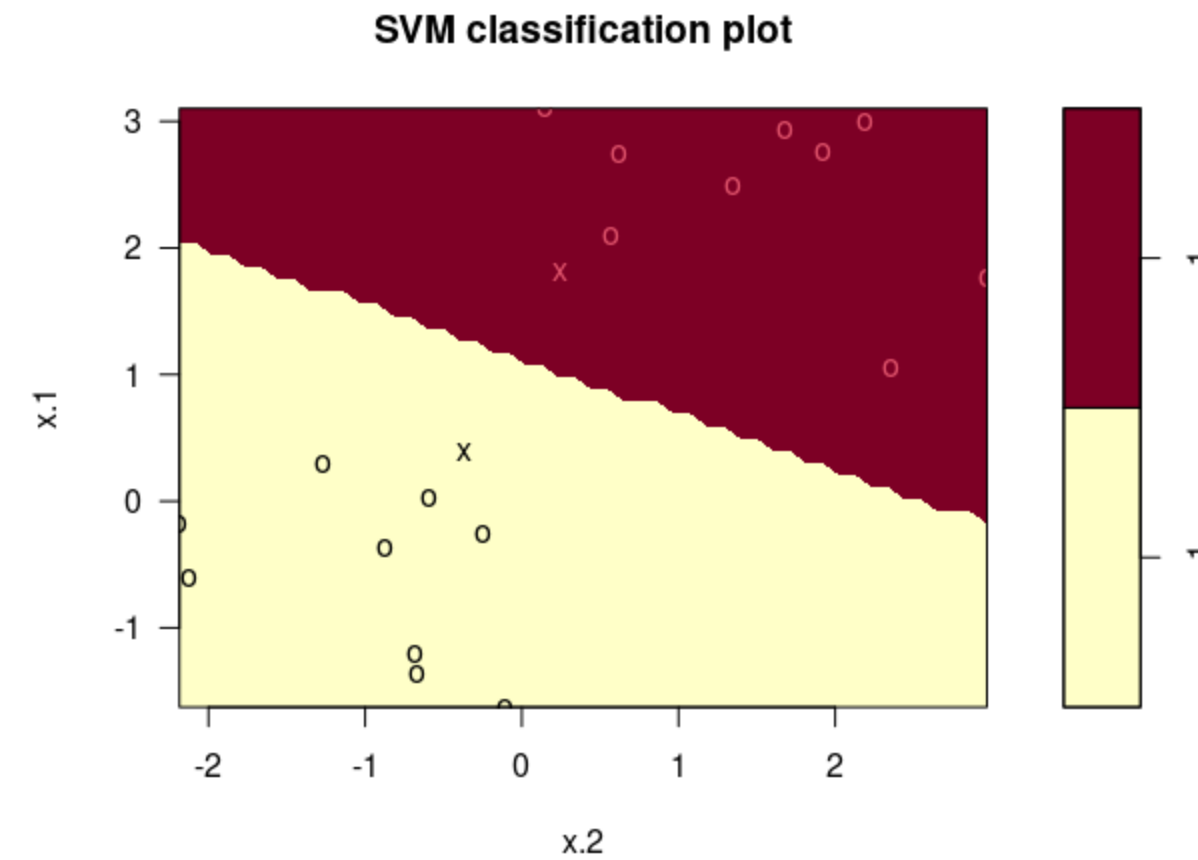
# Plot data
ggplot(
  data = dat,
  aes(x = x.2, y = x.1, color = y, shape = y)
) +
  geom_point(size = 2) +
  scale_color_manual(values=c("#000000", "#FF0000")) +
  theme(legend.position = "none")
```



Classification Problems

SVM Classification using e1071

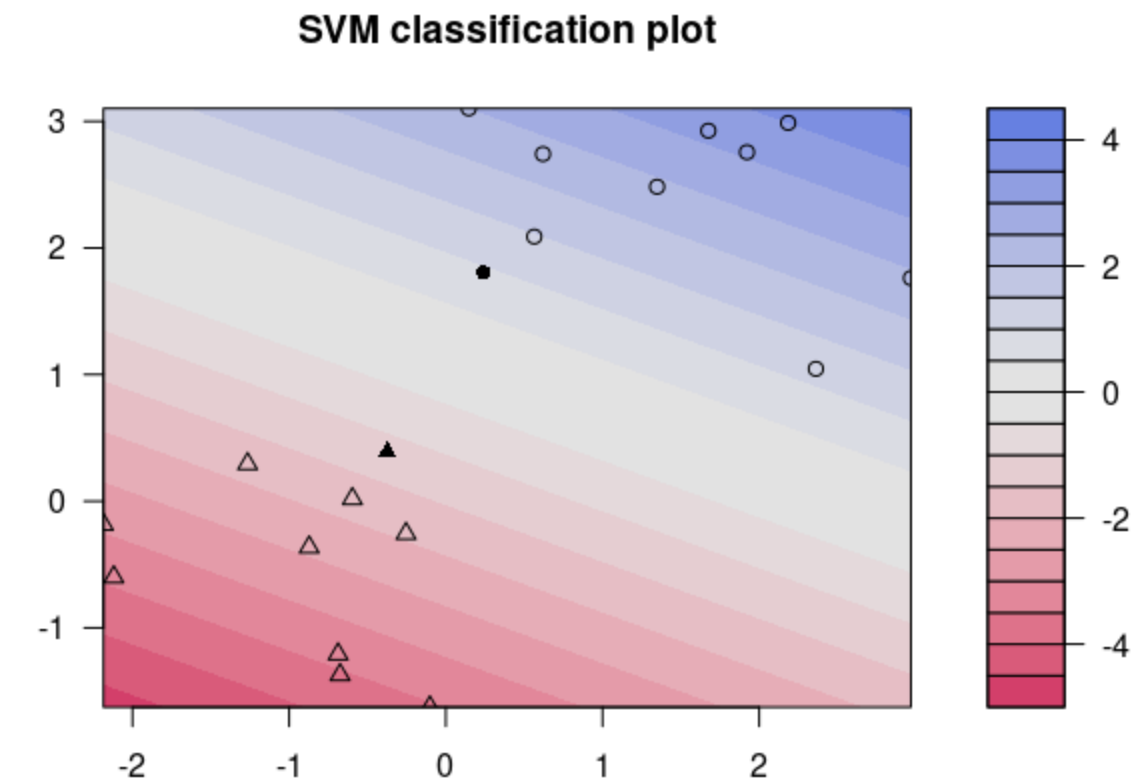
```
# Fit Support Vector Machine model to data set
svmfit <- svm(
  y~., data = dat,
  kernel = "linear", scale = FALSE,
  cost = 1e10
)
# Plot Results
plot(svmfit, dat)
```



Classification Problems

SVM Classification using kernfit

```
# fit model and produce plot
kernfit <- ksvm(
  x, y,
  type = "C-svc", kernel = 'vanilladot',
  C = 1e10
)
plot(kernfit, data = x)
```



Classification Problems

SVM Classification Objects

```
svmfit
Call:
svm(formula = y ~ ., data = dat, kernel = "linear", scale = FALSE)

Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  linear
      cost:  1e+10

Number of Support Vectors:  2
```

```
kernfit
Support Vector Machine object of class "ksvm"

SV type: C-svc (classification)
parameter : cost C = 1e+10

Linear (vanilla) kernel function.

Number of Support Vectors : 2

Objective Function Value : -2.1068
Training error : 0
```

Classification Problems

SVM Classification using sklearn (Python)

```
# Authors: The scikit-learn developers
# SPDX-License-Identifier: BSD-3-Clause

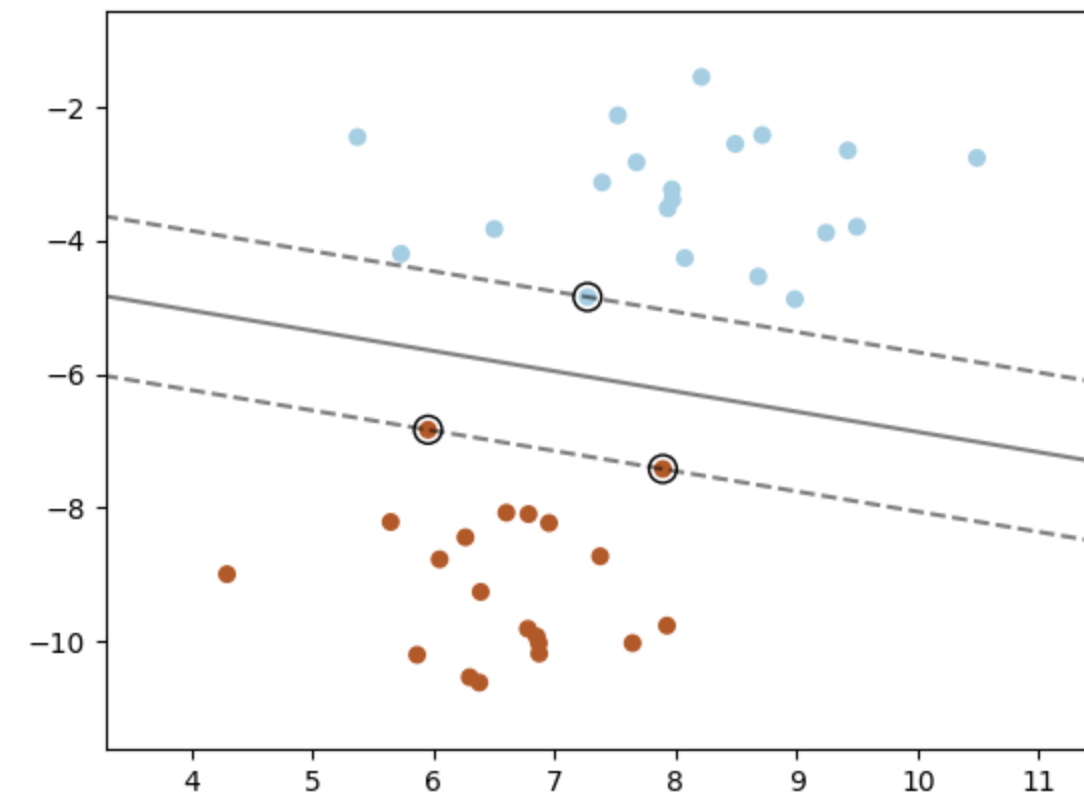
import matplotlib.pyplot as plt

from sklearn import svm
from sklearn.datasets import make_blobs
from sklearn.inspection import DecisionBoundaryDisplay

# we create 40 separable points
X, y = make_blobs(n_samples=40, centers=2, random_state=6)

# fit the model
clf = svm.SVC(kernel="linear", C=1000)
clf.fit(X, y)

plt.scatter(
    X[:, 0], X[:, 1],
    c=y, s=30,
```

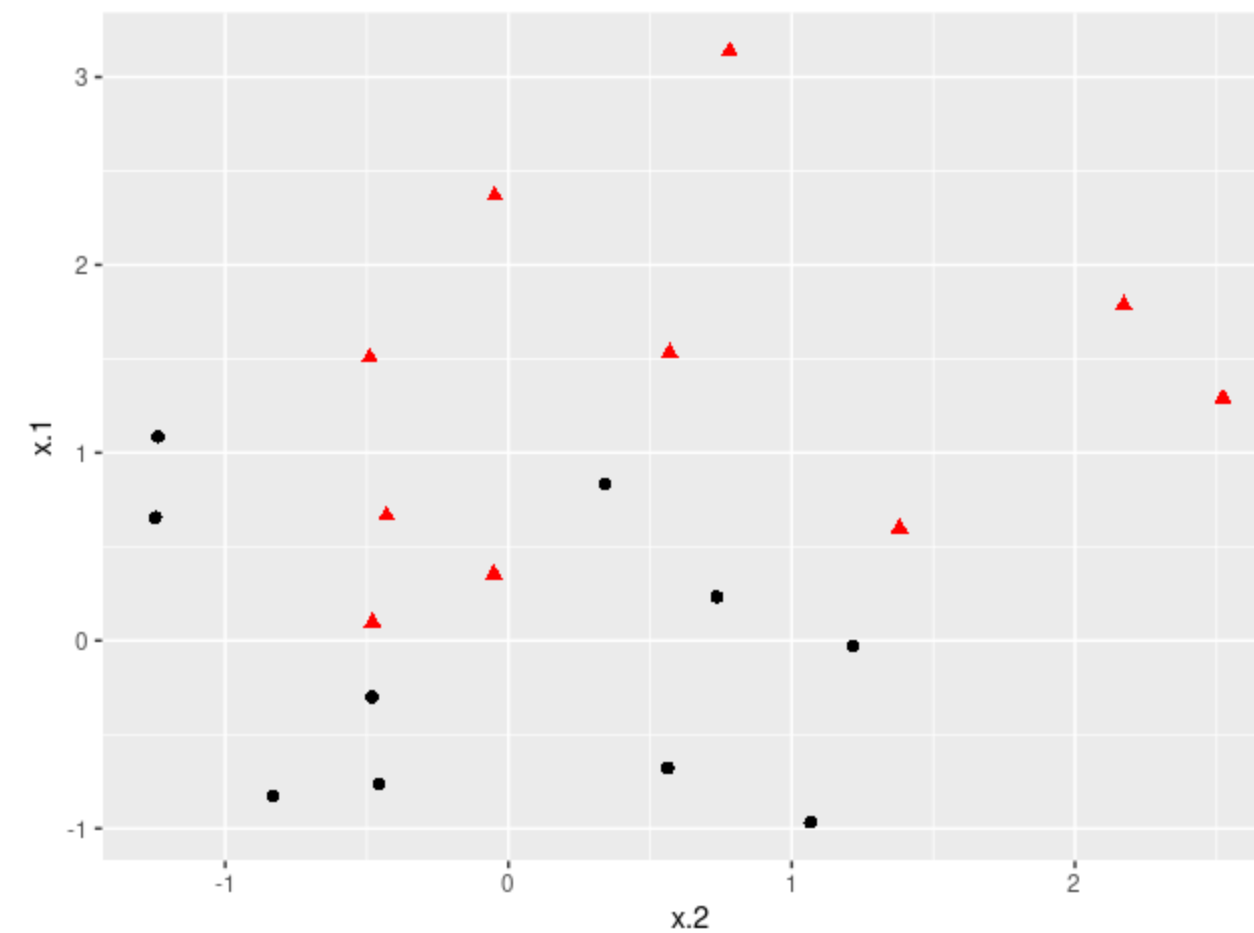


Classification Problems

Random Test Data (partly overlapping)

```
# Construct sample data set - not completely separated
x <- matrix(rnorm(20*2), ncol = 2)
y <- c(rep(-1,10), rep(1,10))
x[y==1,] <- x[y==1,] + 1
dat <- data.frame(x=x, y=as.factor(y))

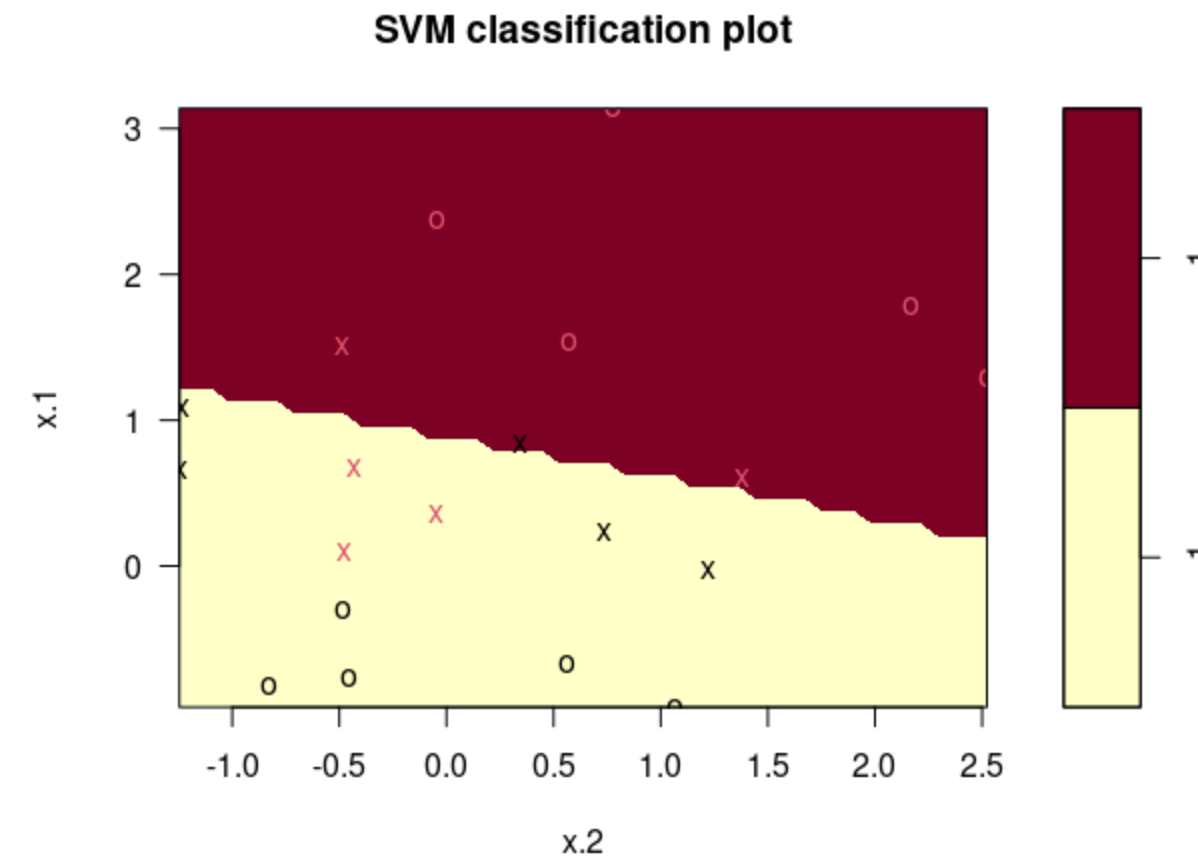
# Plot data set
ggplot(
  data = dat,
  aes(x = x.2, y = x.1, color = y, shape = y)
) +
  geom_point(size = 2) +
  scale_color_manual(values=c("#000000", "#FF0000")) +
  theme(legend.position = "none")
```



Classification Problems

Classification using e1071 (cost = 10)

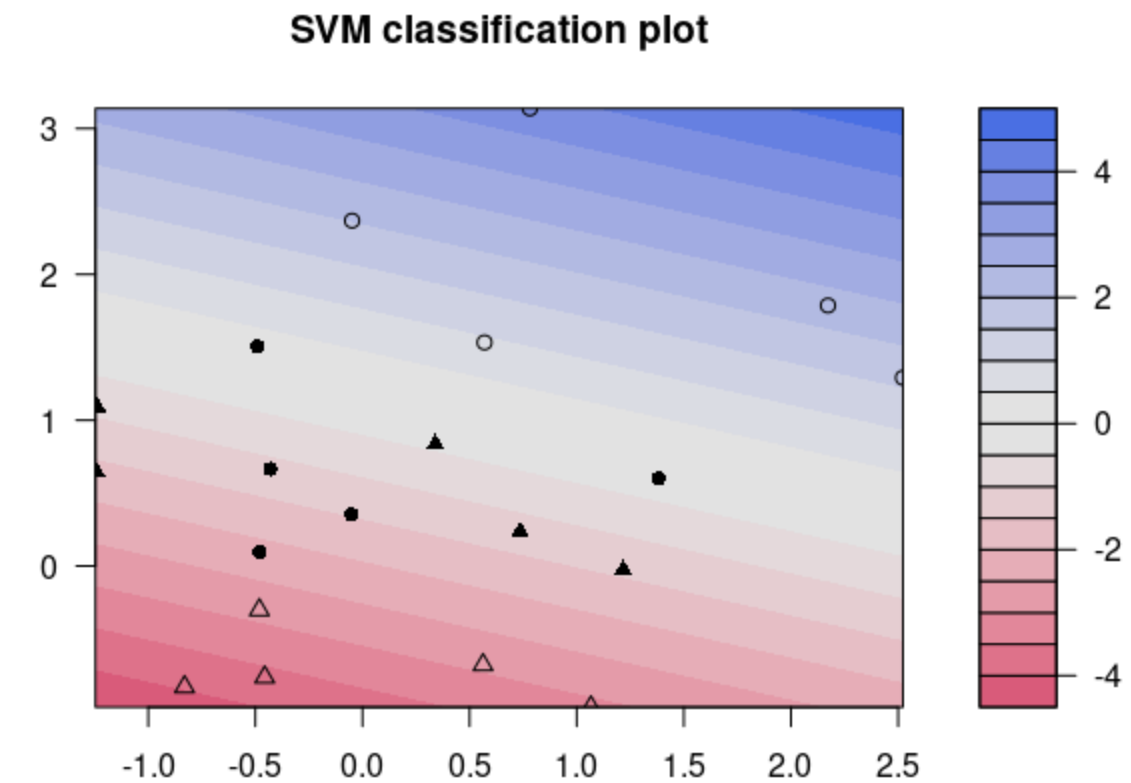
```
# Fit Support Vector Machine model to data set
svmfit <- svm(y~., data = dat, kernel = "linear", cost = 10)
# Plot Results
plot(svmfit, dat)
```



Classification Problems

Classification using `kernfit` (cost = 1000)

```
# fit model and produce plot
kernfit <- ksvm(
  x, y, type = "C-svc",
  kernel = 'vanilladot', C = 1000
)
plot(kernfit, data = x)
```



Classification Problems

Tune the cost hyperparameter

```
tune.out <- tune(
  svm, y~., data = dat, kernel = "linear",
  ranges = list(cost = c(0.001, 0.01, 0.1, 1, 5, 10, 100))
)
(bestmod <- tune.out$best.model)

Call:
best.tune(method = svm, train.x = y ~ ., data = dat, ranges = list(cost = c(0.001,
  0.01, 0.1, 1, 5, 10, 100)), kernel = "linear")

Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  linear
      cost:  0.1

Number of Support Vectors:  16
```

Classification Problems

Analyse Model Performance

```
# Create a table of misclassified observations
ypred <- predict(bestmod, dat)
(misclass <- table(predict = ypred, truth = dat$y))

      truth
predict -1  1
      -1   9  3
       1   1  7
```

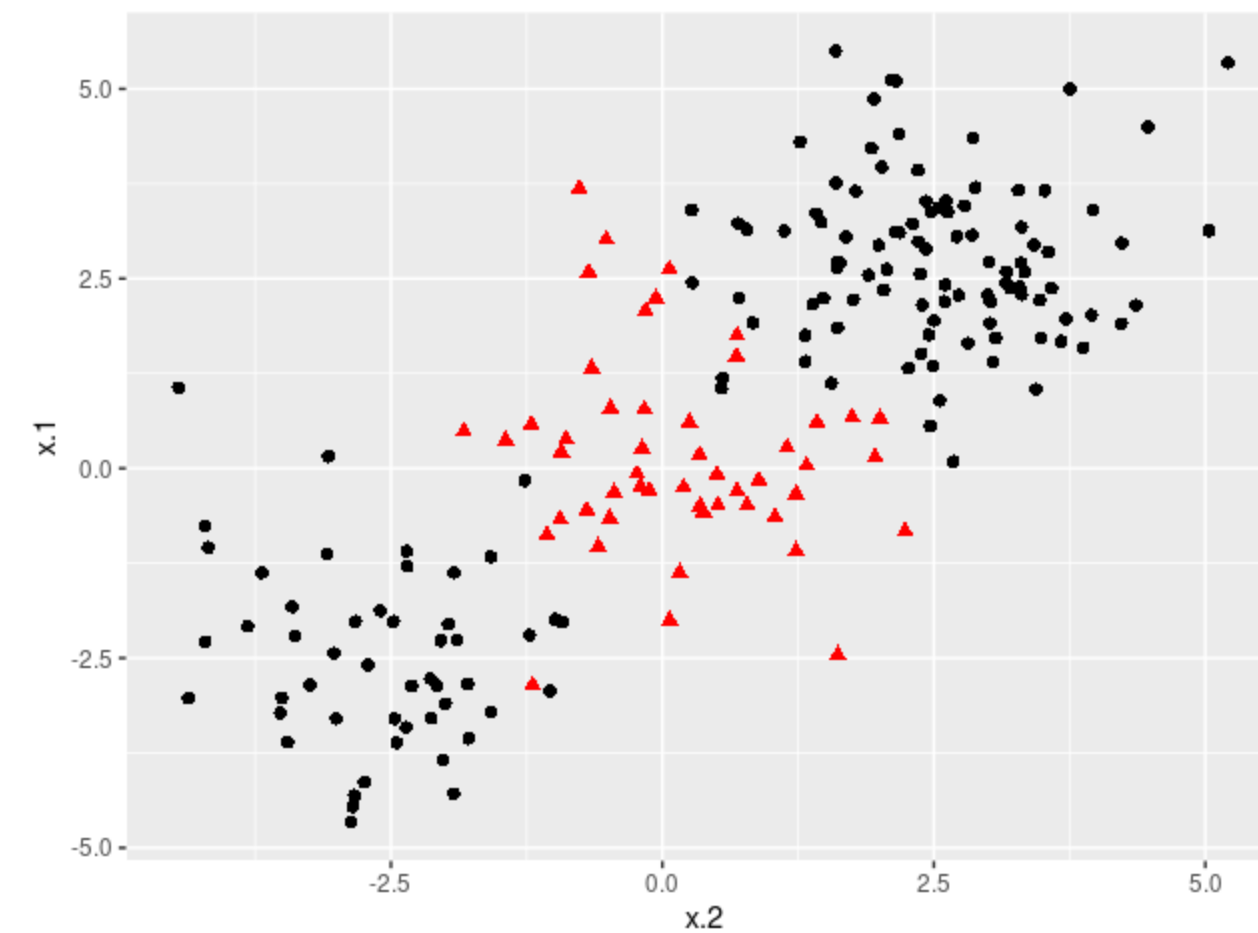
Of course we should really split our data into **training and testing sets**!

Classification Problems

Random Test Data (non linear)

```
# construct larger random data set
x <- matrix(rnorm(200*2), ncol = 2)
x[1:100,] <- x[1:100,] + 2.5
x[101:150,] <- x[101:150,] - 2.5
y <- c(rep(1,150), rep(2,50))
dat <- data.frame(x=x,y=as.factor(y))

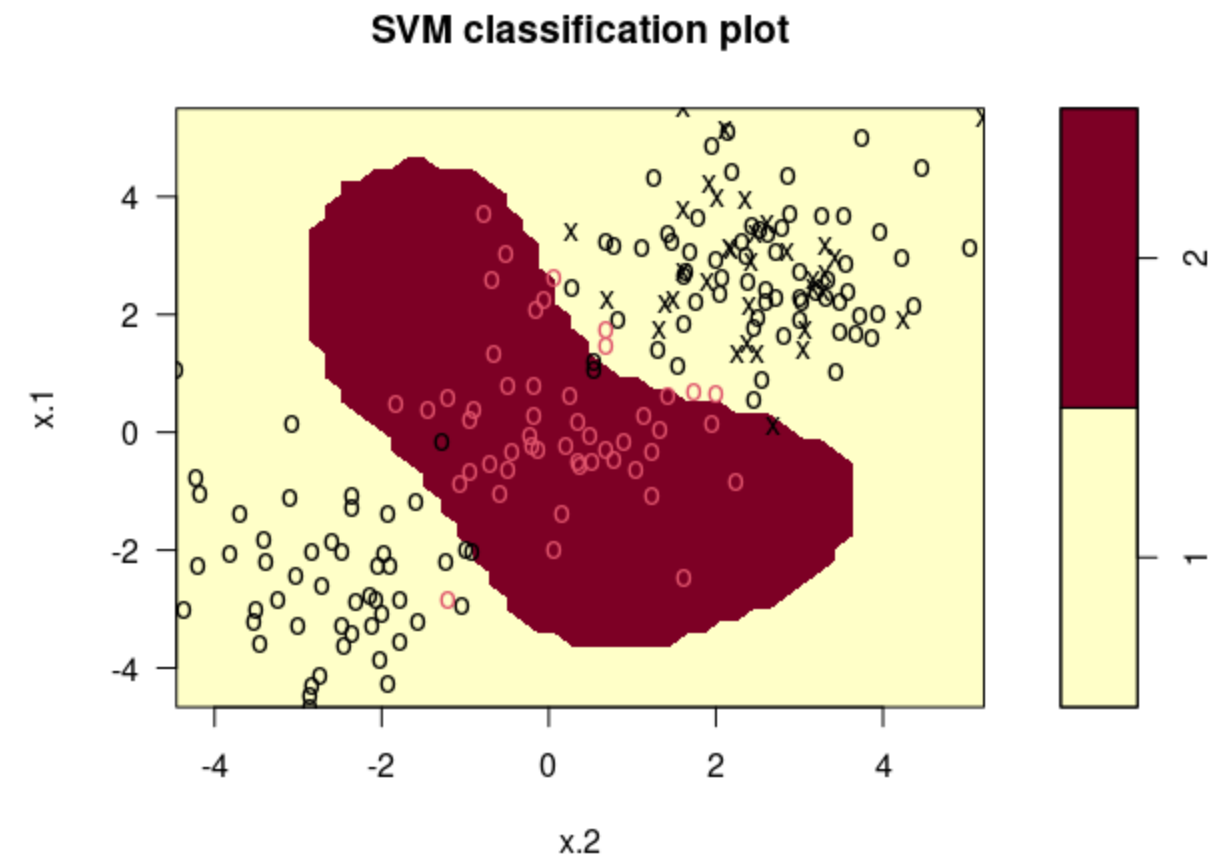
# Plot data
ggplot(
  data = dat,
  aes(x = x.2, y = x.1, color = y, shape = y)
) +
  geom_point(size = 2) +
  scale_color_manual(values=c("#000000", "#FF0000")) +
  theme(legend.position = "none")
```



Classification Problems

Classification using e1071 (radial)

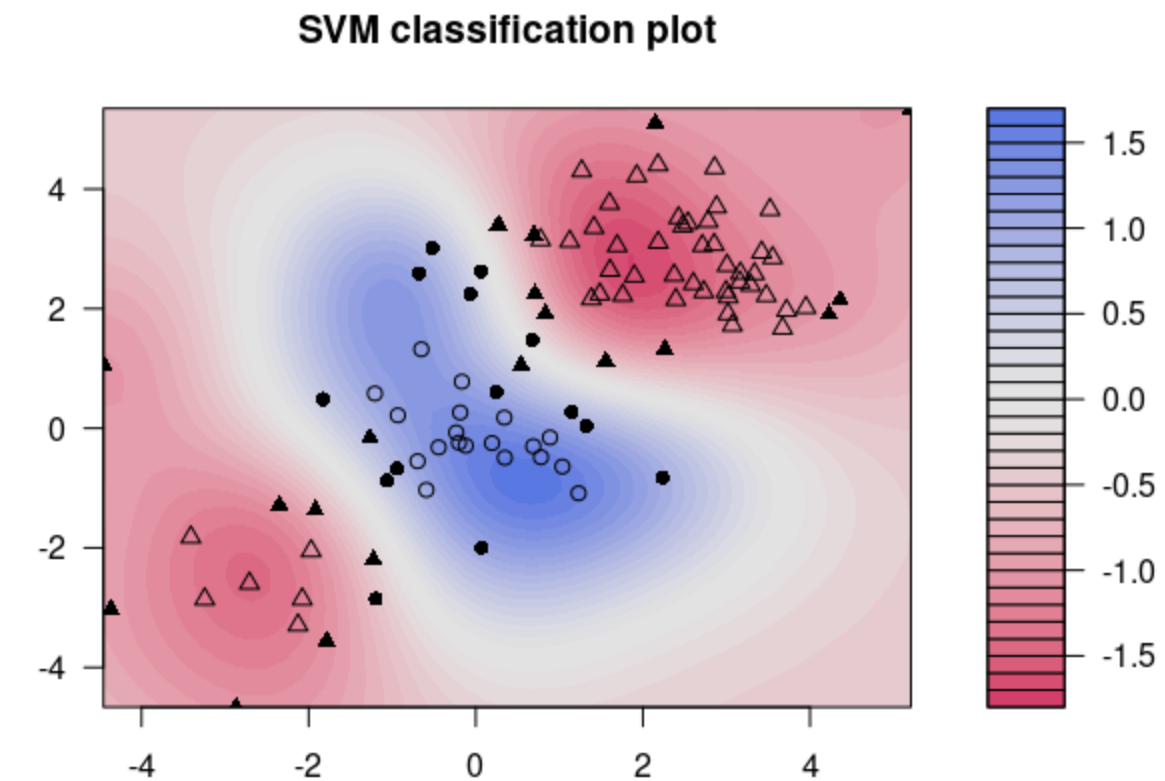
```
set.seed(123)
# sample training data and fit model
train <- base::sample(200,100, replace = FALSE)
svmfit <- svm(
  y~., data = dat[train,],
  kernel = "radial", gamma = 1, cost = 1
)
# plot classifier
plot(svmfit, dat)
```



Classification Problems

Classification using `kernfit` (`rbfdot`)

```
# Fit radial-based SVM in kernlab
kernfit <- ksvm(
  x[train,], y[train],
  type = "C-svc", kernel = 'rbfdot', C = 1, scaled = c()
)
# Plot training data
plot(kernfit, data = x[train,])
```



Classification Problems

Tune the radial model

```
# tune model to find optimal cost, gamma values
tune.out <- tune(svm, y~., data = dat[train,], kernel = "radial",
               ranges = list(cost = c(0.1,1,10,100,1000),
                             gamma = c(0.5,1,2,3,4)))
# show best model
tune.out$best.model

Call:
best.tune(method = svm, train.x = y ~ ., data = dat[train, ], ranges = list(cost = c(0.1,
  1, 10, 100, 1000), gamma = c(0.5, 1, 2, 3, 4)), kernel = "radial")

Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  radial
      cost:  1

Number of Support Vectors:  30
```

Classification Problems

Analyse Model Performance (radial)

```
# validate model performance
(valid <- table(
  true = dat[-train,"y"],
  pred = predict(
    tune.out$best.mode,
    newx = dat[-train,]
  )
))

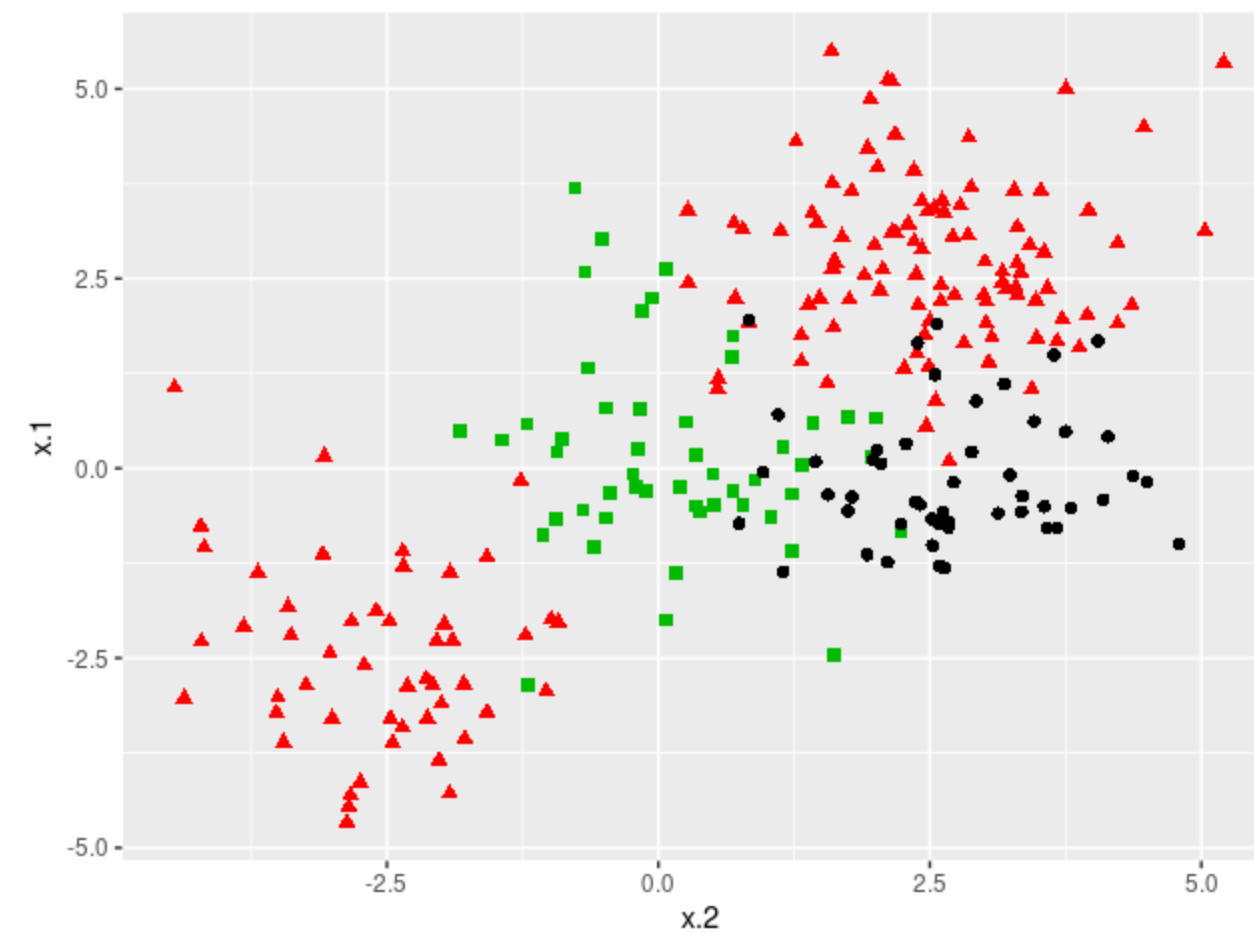
      pred
true   1   2
  1  55  28
  2  12   5
```

This corresponds to an accuracy of around 65%.

Classification Problems

Random Test Data (multiclass)

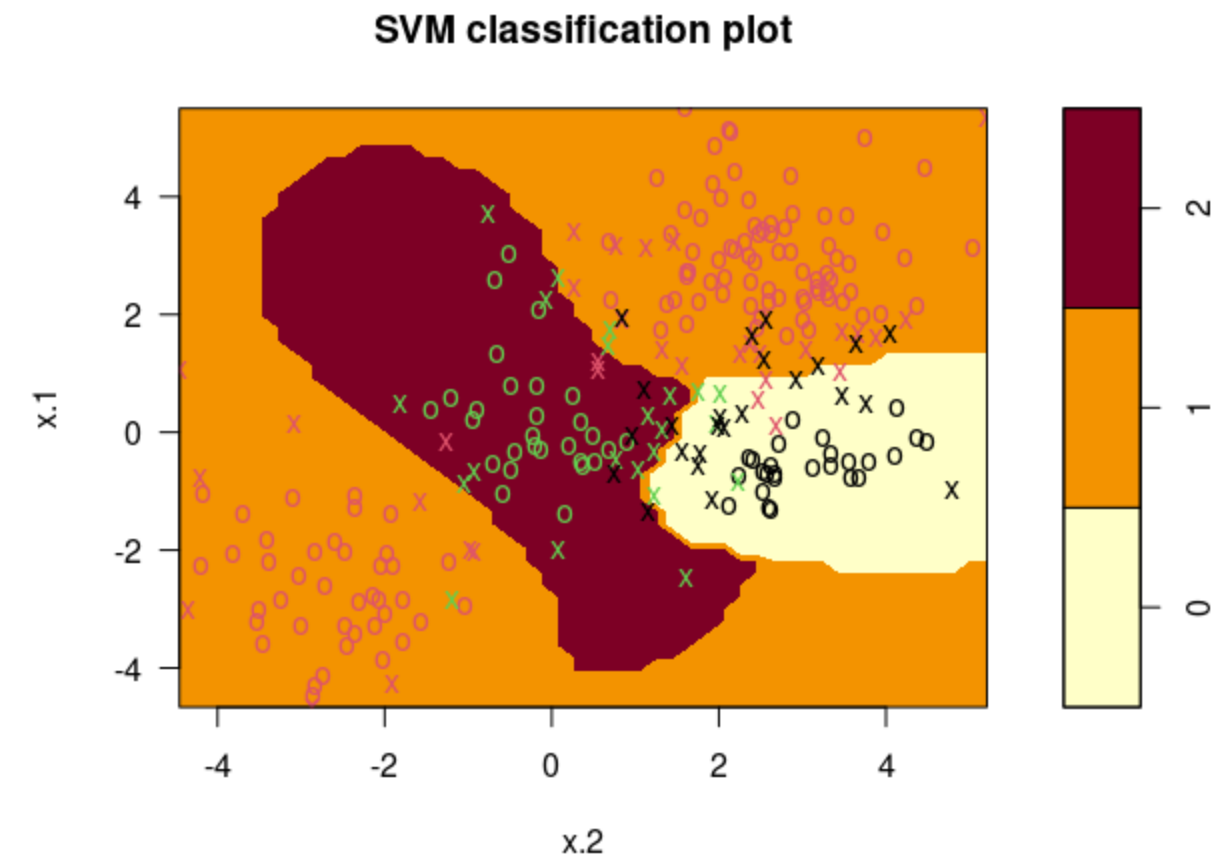
```
# construct data set
x <- rbind(x, matrix(rnorm(50*2), ncol = 2))
y <- c(y, rep(0,50))
x[y==0,2] <- x[y==0,2] + 2.5
dat <- data.frame(x=x, y=as.factor(y))
# plot data set
ggplot(
  data = dat,
  aes(x = x.2, y = x.1, color = y, shape = y)
) +
  geom_point(size = 2) +
  scale_color_manual(values=c("#000000", "#FF0000", "#00BA00"))
  theme(legend.position = "none")
```



Classification Problems

Classification using e1071 (multiclass)

```
# fit model
svmfit <- svm(
  y~., data = dat, kernel = "radial",
  cost = 10, gamma = 1
)
# plot results
plot(svmfit, dat)
```



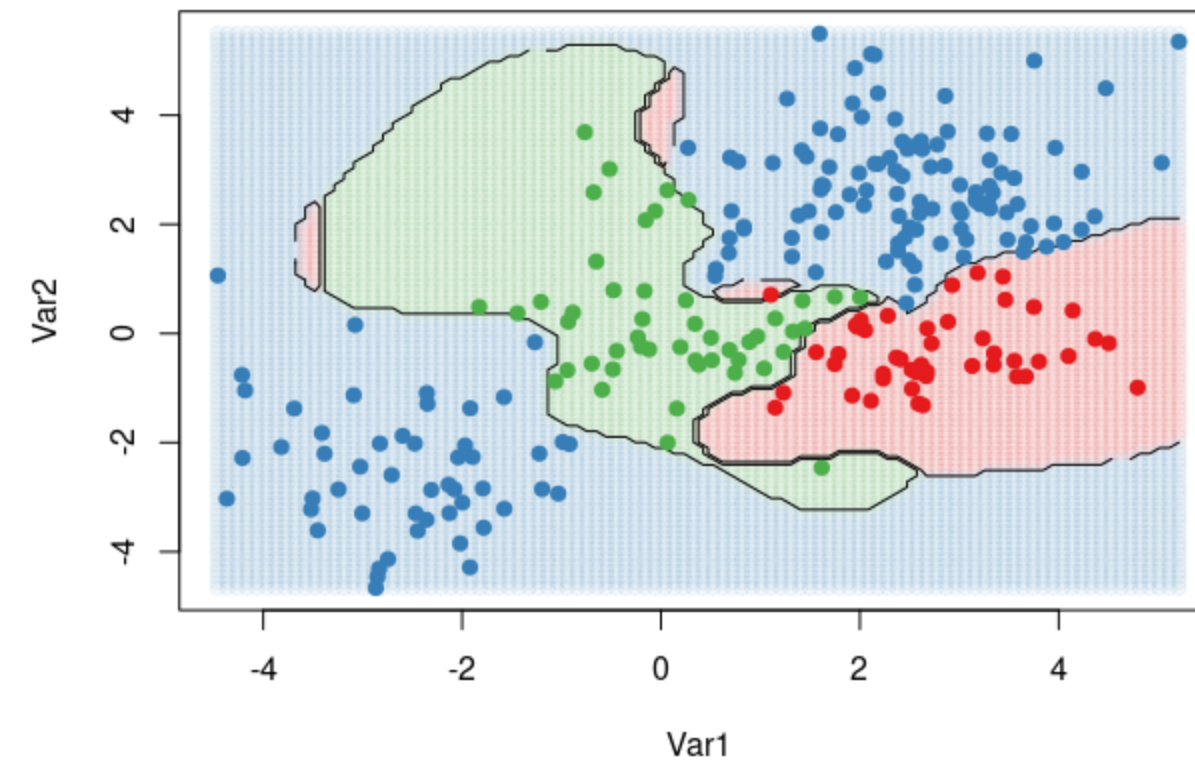
Classification using `kernfit` (multiclass)

```
# fit and plot
kernfit <- ksvm(as.matrix(dat[,2:1]), dat$y,
               type = "C-svc", kernel = 'rbfdot',
               C = 100, scaled = c())

# Create a fine grid of the feature space
x.1 <- seq(min(dat$x.1), max(dat$x.1), 100)
x.2 <- seq(min(dat$x.2), max(dat$x.2), 100)
x.grid <- expand.grid(x.2, x.1)

# Get class predictions over grid
pred <- predict(kernfit, newdata = x.grid)

# Plot the results
cols <- brewer.pal(3, "Set1")
plot(
  x.grid, pch = 19,
  col = adjustcolor(cols[pred], alpha.f = 0.05)
```



Classification Problems

Application (Khan gene sequences)

```
dat <- data.frame(x = Khan$xtrain, y=as.factor(Khan$ytrain))
(out <- svm(y~., data = dat, kernel = "linear", cost = 10))

Call:
svm(formula = y ~ ., data = dat, kernel = "linear", cost = 10)

Parameters:
  SVM-Type:  C-classification
SVM-Kernel:  linear
      cost:  10

Number of Support Vectors:  58
```

```
# check model performance on training set
table(out$fitted, dat$y)
```

	1	2	3	4
1	8	0	0	0
2	0	23	0	0
3	0	0	12	0
4	0	0	0	20

```
# validate model performance
dat.te <- data.frame(x=Khan$xtest, y=as.factor(Khan$ytest))
pred.te <- predict(out, newdata=dat.te)
table(pred.te, dat.te$y)
```

pred.te	1	2	3	4
1	3	0	0	0
2	0	6	2	0
3	0	0	4	0
4	0	0	0	5

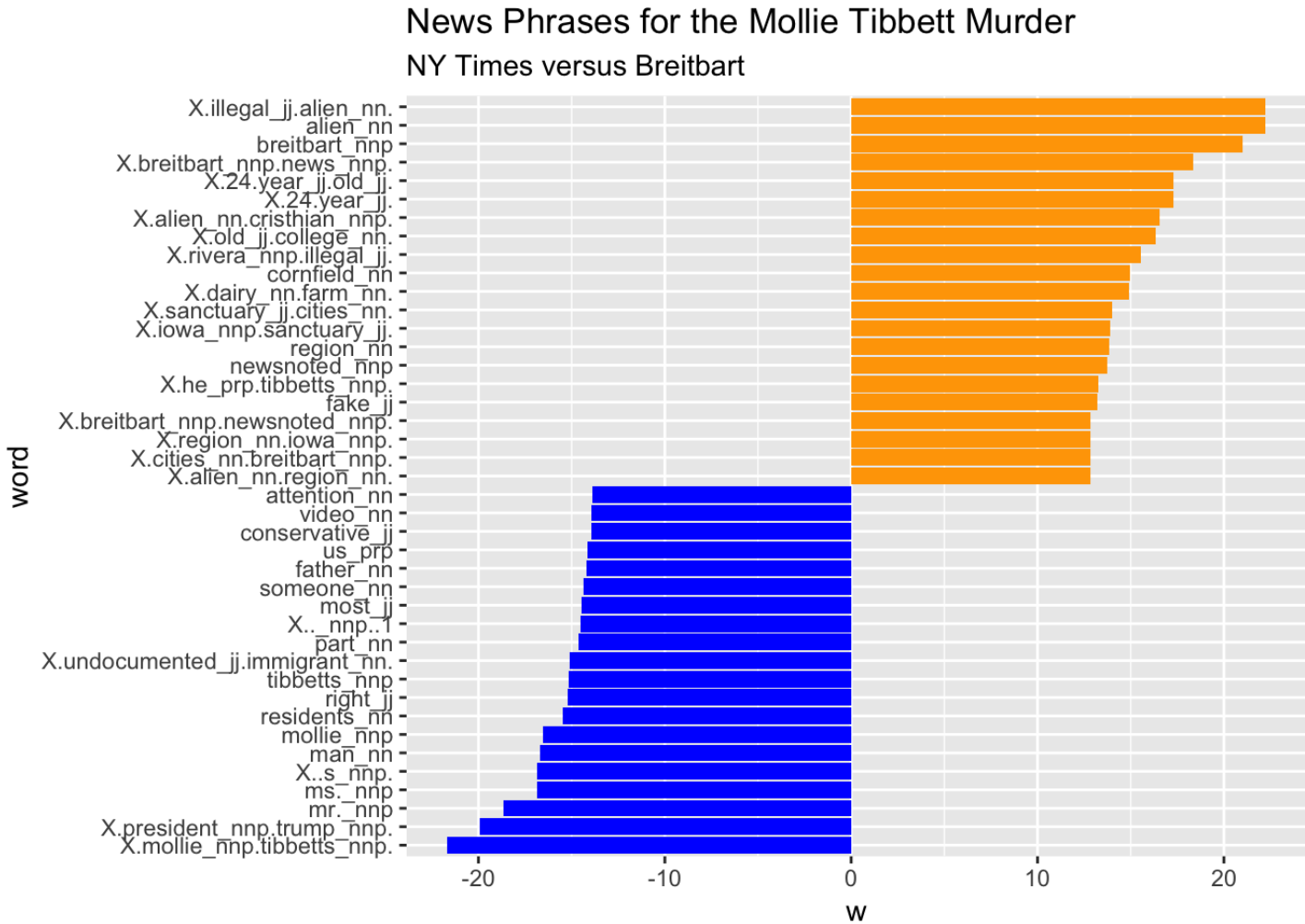


Classification Problems

Application (Text Classification)

Classifying Text by Media Source

Source: [Marisa Smith](#)



Classification Problems

Pros & Cons of SVM

Pros

- It works really well with a clear margin of separation
- It is effective in high dimensional spaces
- It is effective in cases where the number of dimensions is greater than the number of samples
- It uses a subset of training points in the decision function (support vectors), so it is memory efficient

Cons

- It is not always straightforward to scale the different variables "equally"
- It is very prone to overfitting (different Kernel functions, hyperparameters etc.)
- It doesn't perform well when we have large data set because the required training time is higher
- It also doesn't perform very well, when the data set has more noise i.e. target classes are overlapping
- It doesn't directly provide probability estimates

Exercise

***iris* dataset**

Try to find a "good" SVM classification model for the `iris` dataset.

Exercise

Classify Handwritten Letter Data (1)

Preparation

Download the [training](#) dataset and make sure you have the necessary packages installed

```
library(e1071)
```

Data Loading

Load the data and make some transformations

```
digits_train <- read.csv("optdigits.tra", header = FALSE)
digits_train <- data.frame(x = digits_train[,-1], y = as.factor(digits_train[,65]))
```

Look at one digit

```
m = matrix(unlist(digits_train[1,-1]),8,8)
image(m, axes = FALSE, col = grey(seq(0, 1, length = 256)))

digits_train[1,65]
```


Exercise

Classify Handwritten Letter Data (2)

Try a first SVM classification

```
digits_svm <- svm(y~., data = digits_train, kernel = "linear", cost = 10, scale = FALSE)
```

Have a look at the confusion matrix

Does this make sense? What could be the problem? Solve it!

```
table(digits_svm$fitted, digits_train$y)
```

Is there a better kernel? Tune the cost hyperparameter.

Find the optimal value while preventing any overfit.

Classification with Cross Validation

Classification with Cross Validation

Dataset heart

In this example adapted from [here](#) we are going to use the `heart` dataset:

A mixed variable dataset containing 14 variables of 297 patients for their heart disease diagnosis.

```
library(caret)
library(kmed)
data(heart)
str(heart)

'data.frame':   297 obs. of  14 variables:
 $ age      : num  63 67 67 37 41 56 62 57 63 53 ...
 $ sex      : logi  TRUE TRUE TRUE TRUE FALSE TRUE ...
 $ cp       : Factor w/ 4 levels "1","2","3","4": 1 4 4 3 2 2 4 4 4 4 ...
 $ trestbps: num  145 160 120 130 130 120 140 120 130 140 ...
 $ chol     : num  233 286 229 250 204 236 268 354 254 203 ...
 $ fbs      : logi  TRUE FALSE FALSE FALSE FALSE FALSE ...
 $ restecg  : Factor w/ 3 levels "0","1","2": 3 3 3 1 3 1 3 1 3 3 ...
 $ thalach  : num  150 108 129 187 172 178 160 163 147 155 ...
 $ exang    : logi  FALSE TRUE TRUE FALSE FALSE FALSE ...
 $ oldpeak  : num  2.3 1.5 2.6 3.5 1.4 0.8 3.6 0.6 1.4 3.1 ...
 $ slope    : Factor w/ 3 levels "1","2","3": 3 2 2 3 1 1 3 1 2 3 ...
 $ ca      : num  0 3 2 0 0 0 2 0 1 0 ...
 $ thal     : Factor w/ 3 levels "3","6","7": 2 1 3 1 1 1 1 1 3 3 ...
 $ class    : int  0 2 1 0 0 0 3 0 2 1 ...
 - attr(*, "na.action")= 'omit' Named int [1:6] 88 167 193 267 288 303
 ..- attr(*, "names")= chr [1:6] "88" "167" "193" "267" ...
```

Classification with Cross Validation

Train and Test Sets, run CV

```
set.seed(123)
intrain <- createDataPartition(y = heart$class, p= 0.7, list = FALSE)
training <- heart[intrain,]
testing <- heart[-intrain,]
training[["class"]] = factor(training[["class"]] != 0)
testing[["class"]] = factor(testing[["class"]] != 0)
```

```
trctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 3)
svm_linear <- train(class ~., data = training, method = "svmLinear",
  trControl=trctrl,
  preProcess = c("center", "scale"),
  tuneLength = 10)
```

Classification with Cross Validation

Result from first Run

```
Support Vector Machines with Linear Kernel

209 samples
13 predictor
2 classes: 'FALSE', 'TRUE'

Pre-processing: centered (18), scaled (18)
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 189, 188, 188, 188, 188, 188, ...
Resampling results:

    Accuracy    Kappa
0.8421789    0.6750873

Tuning parameter 'C' was held constant at a value of 1
```


Classification with Cross Validation

Apply to Test Data

```
test_pred <- predict(svm_linear, newdata = testing)
test_pred
confusionMatrix(table(test_pred, testing$class))
```

Confusion Matrix **and** Statistics

```
test_pred FALSE TRUE
FALSE     40    16
TRUE       2    30
```

```
Accuracy : 0.7955
 95% CI : (0.6961, 0.874)
No Information Rate : 0.5227
P-Value [Acc > NIR] : 1.042e-07
```

```
Kappa : 0.5959
```

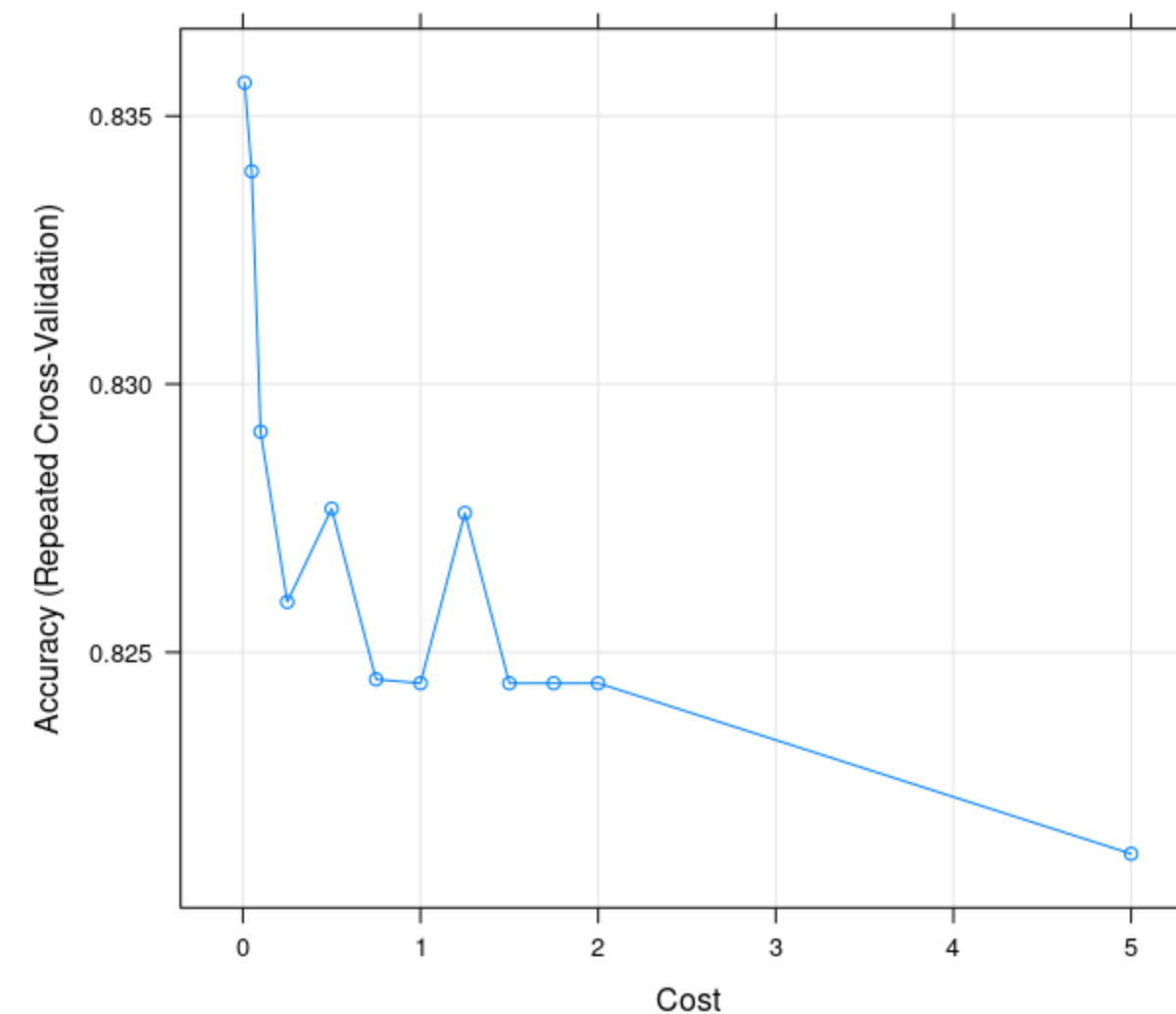
```
McNemar's Test P-Value : 0.002183
```

```
Sensitivity : 0.9524
Specificity : 0.6522
Pos Pred Value : 0.7143
Neg Pred Value : 0.9375
```

Classification with Cross Validation

Tune cost using caret (1)

```
grid <- expand.grid(  
  C = c(0, 0.01, 0.05, 0.1, 0.25, 0.5,  
        0.75, 1, 1.25, 1.5, 1.75, 2, 5))  
svm_linear_grid <- train(  
  class ~., data = training, method = "svmLinear",  
  trControl = trctrl,  
  preProcess = c("center", "scale"),  
  tuneGrid = grid,  
  tuneLength = 10  
)  
plot(svm_linear_grid)
```





Classification with Cross Validation

Tune cost using caret (2)

```
Support Vector Machines with Linear Kernel

209 samples
13 predictor
2 classes: 'FALSE', 'TRUE'

Pre-processing: centered (18), scaled (18)
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 188, 189, 188, 188, 188, 187, ...
Resampling results across tuning parameters:
```

C	Accuracy	Kappa
0.00	NaN	NaN
0.01	0.8356205	0.6609027
0.05	0.8339683	0.6587163
0.10	0.8291126	0.6498767
0.25	0.8259380	0.6451992
0.50	0.8276768	0.6469223
0.75	0.8244949	0.6403443
1.00	0.8014000	0.6100000

Classification with Cross Validation

Optimized Model on Test Data

```
test_pred_grid <- predict(svm_linear_grid, newdata = testing)
test_pred_grid
confusionMatrix(table(test_pred_grid, testing$class))
```

Confusion Matrix **and** Statistics

```
test_pred_grid FALSE TRUE
      FALSE    42    15
      TRUE     0    31
```

```
      Accuracy : 0.8295
      95% CI : (0.7345, 0.9013)
No Information Rate : 0.5227
P-Value [Acc > NIR] : 1.673e-09
```

```
      Kappa : 0.6636
```

```
McNemar's Test P-Value : 0.0003006
```

```
      Sensitivity : 1.0000
      Specificity : 0.6739
Pos Pred Value : 0.7368
Neg Pred Value : 1.0000
```

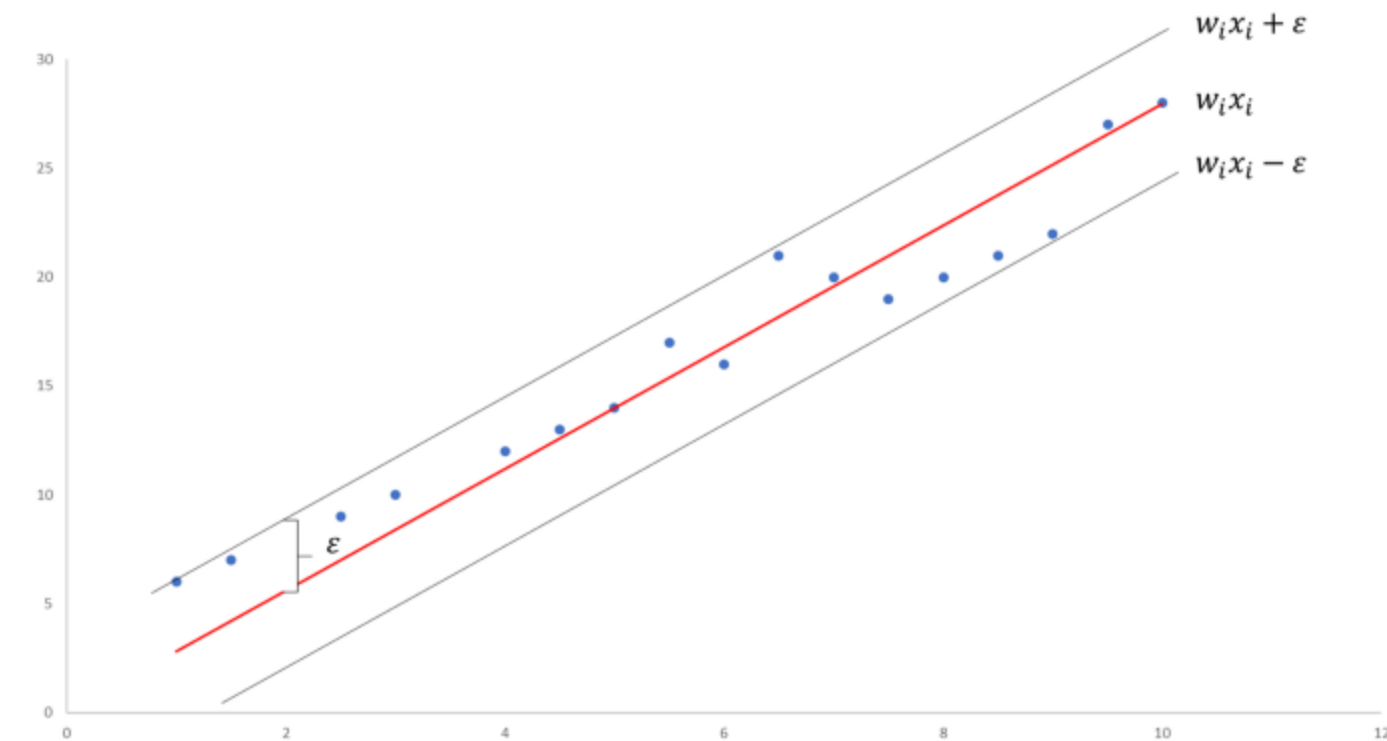
Support Vector Regression

Support Vector Regression

Basic Idea

SVR gives us the flexibility to define how much error is acceptable in our model and will find an appropriate line (or hyperplane in higher dimensions) to fit the data. In contrast to OLS, the objective function of SVR is to **minimize the coefficients** not the squared error.

The error term is instead handled in the constraints, where we set the absolute error less than or equal to a specified margin, called the maximum error.



Support Vector Regression

Dataset Boston

For the regression example (adapted from [here](#)) we use the Boston housing data

```
library(e1071)
library(caret)

# Regression example
boston <- MASS::Boston
str(boston)

'data.frame':   506 obs. of  14 variables:
 $ crim      : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
 $ zn        : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
 $ indus     : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
 $ chas      : int   0 0 0 0 0 0 0 0 0 0 ...
 $ nox       : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
 $ rm        : num  6.58 6.42 7.18 7 7.15 ...
 $ age       : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
 $ dis       : num  4.09 4.97 4.97 6.06 6.06 ...
 $ rad       : int   1 2 2 3 3 3 5 5 5 5 ...
 $ tax       : num  296 242 242 222 222 222 311 311 311 311 ...
 $ ptratio   : num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
 $ black     : num  397 397 393 395 397 ...
 $ lstat     : num  4.98 9.14 4.03 2.94 5.33 ...
 $ medv      : num  24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

Support Vector Regression

Support Vector Regression

```
set.seed(123)
indexes <- createDataPartition(boston$medv, p = .9, list = F)
train <- boston[indexes, ]
test <- boston[-indexes, ]
```

```
model_reg <- svm(medv~., data=train)
print(model_reg)
```

```
Call:
svm(formula = medv ~ ., data = train)
```

```
Parameters:
  SVM-Type:  eps-regression
SVM-Kernel:  radial
    cost:    1
   gamma:    0.07692308
  epsilon:    0.1
```

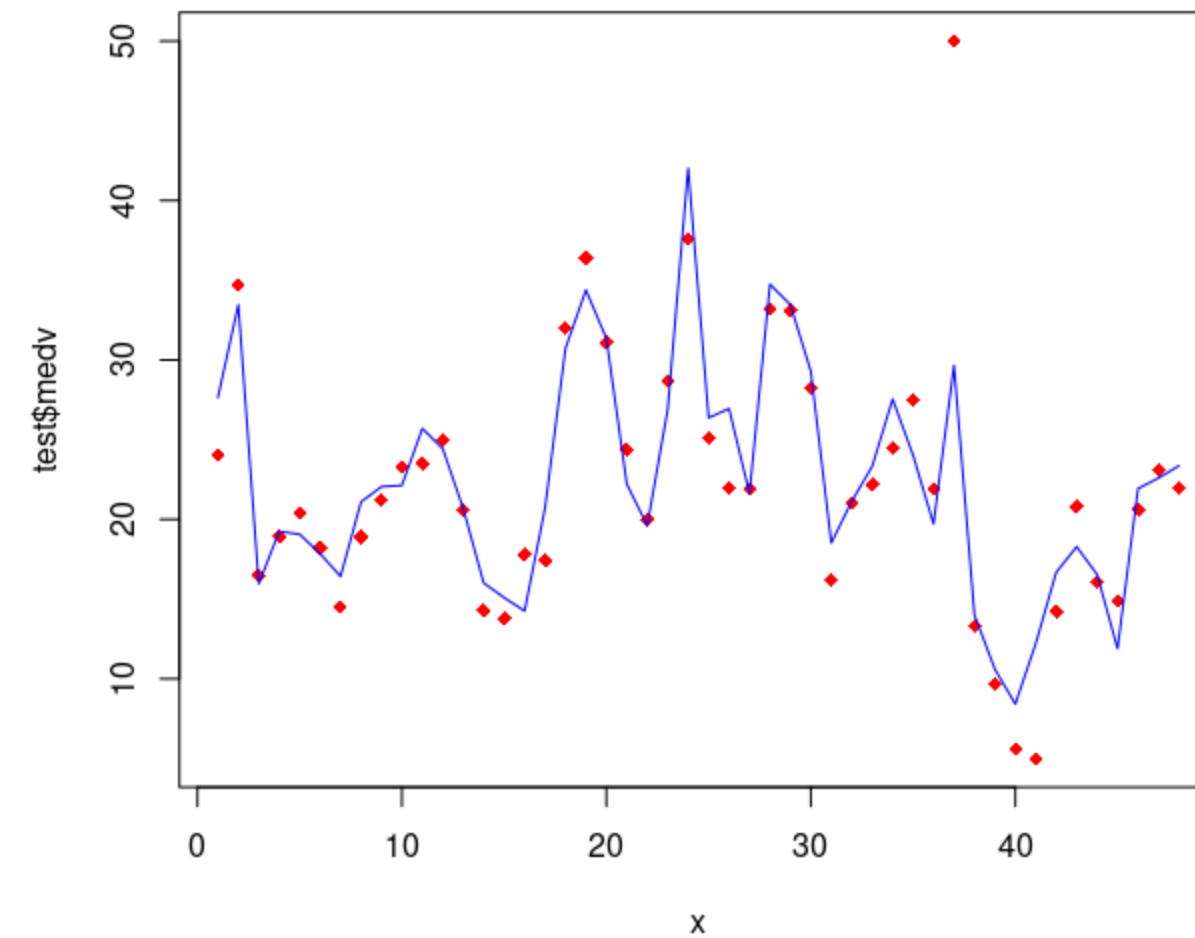
```
Number of Support Vectors: 302
```


Support Vector Regression

Predict Test Data

```
pred <- predict(model_reg, test)
x <- 1:length(test$medv)
plot(x, test$medv, pch=18, col="red")
lines(x, pred, lwd="1", col="blue")
```

```
sqrt(mean((test$medv - pred)^2))
3.701961
```





Support Vector Regression

Look at Outliers

```
test[30:40,]
```

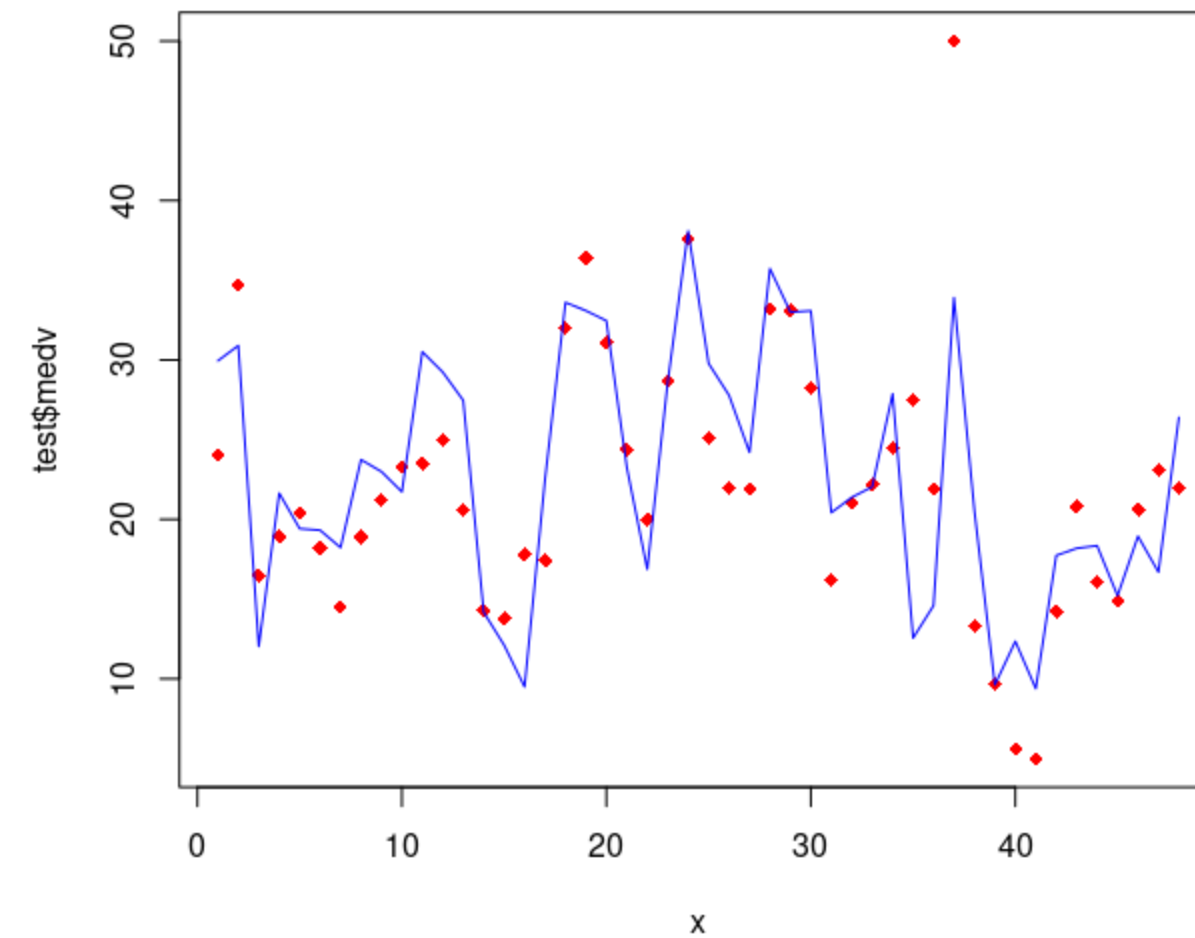
	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
308	0.04932	33	2.18	0	0.472	6.849	70.3	3.1827	7	222	18.4	396.90	7.53	28.2
316	0.25356	0	9.90	0	0.544	5.705	77.7	3.9450	4	304	18.4	396.42	11.50	16.2
320	0.47547	0	9.90	0	0.544	6.113	58.8	4.0019	4	304	18.4	396.23	12.73	21.0
334	0.05083	0	5.19	0	0.515	6.316	38.1	6.4584	5	224	20.2	389.71	5.68	22.2
349	0.01501	80	2.01	0	0.435	6.635	29.7	8.3440	4	280	17.0	390.94	5.99	24.5
366	4.55587	0	18.10	0	0.718	3.561	87.9	1.6132	24	666	20.2	354.70	7.12	27.5
367	3.69695	0	18.10	0	0.718	4.963	91.4	1.7523	24	666	20.2	316.03	14.00	21.9
371	6.53876	0	18.10	1	0.631	7.016	97.5	1.2024	24	666	20.2	392.05	2.96	50.0
378	9.82349	0	18.10	0	0.671	6.794	98.8	1.3580	24	666	20.2	396.90	21.24	13.3
393	11.57790	0	18.10	0	0.700	5.036	97.0	1.7700	24	666	20.2	396.90	25.68	9.7
401	25.04610	0	18.10	0	0.693	5.987	100.0	1.5888	24	666	20.2	396.90	26.77	5.6

Support Vector Regression

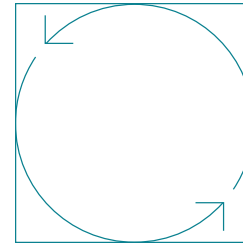
Compare to OLS

```
model_lm <- lm(medv~., data=train)
pred_lm <- predict(model_lm, test)
plot(x, test$medv, pch=18, col="red")
lines(x, pred_lm, lwd="1", col="blue")
```

```
sqrt(mean((test$medv - pred_lm)^2))
5.045893
```



Summary



Support Vector Machines

Summary

- a Support Vector Machine (SVM) is a **non-probabilistic binary linear classifier**
 - can handle non-linear boundaries with the Kernel function/trick
 - can be extended to more than two classes as well
- it is also possible to do regression using this technique (Support Vector Regression)
 - particularly useful in cases where we want to ignore errors up to a certain tolerance
- be careful with scaling your data and overfitting!

October 16th, 2025

Thank you for your attention!

Daniel Meister

+41 44 289 92 30

daniel.meister@datahouse.ch

Datahouse AG

Alte Börse

Bleicherweg 5

CH-8001 Zürich

www.datahouse.ch

Sources and Further Readings

Sources and Further Readings

Support Vector Machines

https://en.wikipedia.org/wiki/Support_vector_machine

Conceptual Introductions

<https://www.edureka.co/blog/support-vector-machine-in-r/>

Classification Tutorials

<http://uc-r.github.io/svm>

Support Vector Regressions

<https://www.svm-tutorial.com/2014/10/support-vector-regression-r/>

<https://www.kdnuggets.com/2017/03/building-regression-models-support-vector-regression.html>